

ALGORITHM OPTIMIZATION OF SHORTEST VECTOR PROBLEMS

最短向量问题的算法优化

FINAL REPORT

SUBMITTED TO SCHOOL OF MATHEMATICS & PHYSICS

OF XI'AN JIAOTONG-LIVERPOOL UNIVERSITY

IN PARTIAL FULFILMENT FOR THE AWARD OF THE DEGREE OF

BSC FINANCIAL MATHEMATICS

By

Chang Liu 2252377

Supervisor: Dr. Keqin Liu

May 19, 2026



Abstract

English Abstract

The Shortest Vector Problem (SVP) is a fundamental computational problem in lattice theory and lattice-based cryptography. It plays an important role in the security assessment of Post-Quantum Cryptography (PQC). Since modern financial systems heavily rely on cryptographic algorithms for secure communication, digital signatures, transaction authentication and other security mechanisms, the migration to post-quantum cryptography can also be regarded as a long-term risk management problem in financial systems. Classical lattice reduction algorithms, such as LLL and traditional BKZ, are commonly used to improve the quality of lattice bases before solving SVP-related problems. However, traditional reduction strategies usually depend on fixed or manually designed algorithmic evolution procedures, making it difficult to adapt to the mathematical structure of different lattices.

This thesis investigates a local BKZ reduction optimization framework based on Deep Reinforcement Learning (DRL). Instead of directly feeding the raw integer lattice basis matrix into the neural network, the state space is reparameterized into a standardized, more informative and dimension-consistent structure. The main purpose of this state space is to improve numerical stability and strengthen the mathematical representation of the lattice basis. The state observed by the agent consists of the cosine matrix of angles between basis vectors, the basis-vector length profile, the logarithmic orthogonality defect, the maximum cosine value and other scalar indicators.

At each step, the agent selects a local BKZ action, including the block size and the reduction position. The backend uses BKZ 2.0 together with periodic LLL for stabilization. The neural architecture adopts an Axial Convolutional Double DQN with noisy exploration. The Axial CNN is used to extract directional features from the cosine matrix, since the rows and columns of the cosine matrix naturally correspond to the indices of basis vectors. The reward function jointly considers the shortening of the first vector, the improvement of global orthogonality, the reduction of the maximum cosine value, the computational cost and additional special reward terms.

Experiments are conducted on the SVP Challenge dataset with multiple random seeds. The reduction quality is evaluated using the ratio between the first vector norm and the Gaussian Heuristic, $\|b_1\|/GH(\mathcal{L})$, the logarithmic orthogonality defect, the maximum cosine value and heatmaps of the cosine matrix. The experimental results show that DRL-based local BKZ scheduling can serve as an adaptive alternative to fixed reduction strategies and can improve

lattice basis quality under a limited number of reduction operations. Future work will further explore multi-dimensional joint training, continued orthogonality optimization after reaching the short-vector target and integration with sieving-assisted BKZ strategies.

Chinese Abstract

最短向量问题 (Shortest Vector Problem, SVP) 是格理论与格密码学中的基础性计算问题, 在后量子密码 (Post-Quantum Cryptography, PQC) 的安全性评估中具有重要作用。由于现代金融系统高度依赖密码学的算法加密, 例如安全通信、数字签名和交易认证等, 后量子密码迁移也可以被视为金融系统中一类长期存在的风险管理问题。经典格基约化算法, 如 LLL 和传统 BKZ, 常被用于在求解 SVP 相关问题前改善格基质量。然而, 传统约化策略通常依赖固定或人工设计的算法演化方式, 难以根据不同格的数学结构自适应调整。

本文研究了一种基于深度强化学习 (Deep Reinforcement Learning, DRL) 的局部 BKZ 约化优化框架。本文将状态空间重参数化为一种正则化、信息更明确且维度一致的结构, 用于代替直接将整数格基矩阵输入神经网络。本文状态设计的主要目的在于提升数值稳定性并强化格基的数学结构表达。智能体的状态空间由基向量的夹角余弦矩阵、基向量长度轮廓、对数正交缺陷、最大余弦值以及其他标量指标组成。

智能体在每一步选择局部 BKZ 动作, 包括块的大小和约化位置, 后端使用 BKZ 2.0 并结合周期性 LLL 稳定化。采用带噪声探索的 Axial Convolutional Double DQN 架构。Axial CNN 用于从余弦矩阵中提取方向性特征, 因为余弦矩阵的行与列天然对应格基向量的索引。奖励函数综合考虑第一向量缩短、全局正交性改善、最大余弦值降低、计算成本以及特殊奖励项。

实验基于多个随机种子的 SVP Challenge 数据集进行。本文使用第一向量范数与高斯启发式之比 $\|b_1\|/GH(\mathcal{L})$ 、对数正交缺陷、最大余弦值以及余弦矩阵热力图评估约化效果。实验结果表明, 基于深度强化学习的局部 BKZ 调度能够作为固定约化策略的自适应替代方案, 并在有限约简次数下改善格基质量。未来工作将进一步探索多维度联合训练、达成短向量目标后继续正交性优化以及与筛法辅助 BKZ 策略的结合。

KEY WORDS: Shortest Vector Problem (SVP), Lattice Basis Reduction, Deep Reinforcement Learning (DRL), BKZ 2.0, Axial CNN, Double DQN-noisy, Cosine Matrix, Orthogonality Defect, Gaussian Heuristic

Acknowledgements

I would like to express my sincere gratitude to my supervisor Dr. Keqin Liu for his invaluable guidance, insightful suggestions and continuous support throughout the development of this thesis. His expertise, patience and rigorous academic standards have been instrumental in shaping both the direction and quality of this work. I would also like to express my sincere appreciation to Dr. Wenbin Zhang. Although he was not my official supervisor, he generously provided valuable advice and guidance on cryptography, which greatly helped me understand the cryptographic background of this project.

I would also like to thank my classmates and friends, Xingyue Fan, Yongrun Huang and Jialun Luo, for their generous assistance and stimulating discussions. Their support in understanding obscure mathematical theories and overcoming programming challenges was very helpful during the completion of this project.

I am deeply grateful to my family for their unconditional love, encouragement and support. Their trust and understanding have been an important source of strength throughout my academic journey.

Finally, I would like to acknowledge Xi'an Jiaotong-Liverpool University (XJTLU) for providing the GPU computing resources and research facilities that made the empirical part of this thesis possible.

Contents

Contents	i
List of Figures	iii
List of Tables	iv
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Core Contributions	2
1.4 Notations	3
1.5 Thesis Organization	4
2 Literature Review	6
2.1 Classical Lattice Reduction: LLL, BKZ and Practical Improvements	6
2.2 Basis Quality, Gaussian Heuristic and Orthogonality	7
2.3 AI-Based Lattice Reduction and Its Difficulties	8
2.4 DRL Architectures for Adaptive BKZ Scheduling	8
2.5 Research Gap	9
3 Methodology	10
3.1 Overview of the DRL-Controlled BKZ Framework	10
3.2 Mathematical Quantities and State Representation	11
Norm/GH Ratio	11
Cosine Matrix	12
Cosine-Token Matrix Used by CNN	12
Gram-Schmidt Length Profile	13
Orthogonality Defect	13
Scalar Indicators	14
Why the Raw Basis Is Not Used	15
3.3 Mathematical Properties of the State Representation	15
Orthogonality Defect and Hadamard Inequality	15
Geometric Information and Limitations of Absolute Cosine	17
Feature Extraction Complexity	18
3.4 Agent Design and Learning Procedure	18

MDP Formulation	18
Reward and Penalty Function	19
AxialCNN-DDQN Architecture	21
NoisyNet Exploration	23
Double DQN Output	24
3.5 Chapter Summary	24
4 Implementation	25
4.1 Chapter Overview	25
4.2 Implementation Pipeline and Data exchange	25
4.3 Cached LLL Initialization	27
4.4 Environment Execution and State Construction	27
4.5 Training Configuration	28
4.6 Baselines, Evaluation and Logging	29
4.7 Experimental Protocol	30
4.8 Chapter Summary	30
5 Experimental Results and Brief Analysis	31
5.1 Training Dynamics	31
5.2 Initial LLL and Final DRL-BKZ Comparison	33
5.3 Full DRL-BKZ Results	34
5.4 Orthogonality Metrics	36
5.5 Cosine Heatmap Comparison	38
5.6 Scope of Available Results	40
5.7 Chapter Summary	40
6 Discussion and Future Work	41
6.1 Discussion and Limitations	41
Overall Interpretation	41
Experimental and Logging Limitations	41
Residual Local Correlation	42
6.2 Future Work	42
Post-target Orthogonality Optimization	42
Dispersed Block and Row Selection	42
Time-series Reward Modelling	43
Multi-dimensional Training	43
Sieving-assisted BKZ	43
7 Conclusion	44
Bibliography	45

List of Figures

4.1	Code-level implementation pipeline.	25
5.1	Best seed's learning curve per dimension.	32
5.2	Experimental success count during training.	33
5.3	Mean Ratio ($\mathbb{E}[\rho(\mathbf{B})]$) Comparison between initial LLL and final DRL-BKZ. . .	34
5.4	DRL-BKZ final ratios with standard deviation.	35
5.5	Per-seed final DRL-BKZ ratios. Each point corresponds to one seed.	36
5.6	Mean logarithmic orthogonality defect ($\mathbb{E}[\delta(\mathbf{B})]$) before and after DRL-BKZ reduction.	37
5.7	Mean maximum cosine before and after DRL-BKZ reduction.	38
5.8	Representative cosine-heatmap comparison among Initial LLL, Fixed Local-BKZ-40 and Final DRL-BKZ.	39

List of Tables

1.1	Mathematical Notations	4
4.1	Data transferred between implementation modules.	26
5.1	Main results of the DRL-BKZ framework under limited computing resource. . . .	34

Chapter 1

Introduction

1.1 Background

Lattice-based cryptography has emerged as a cornerstone of Post-Quantum Cryptography (PQC), offering security guarantees based on worst-case hardness assumptions. As quantum computing capabilities advance, specifically through Shor’s algorithm [16], traditional number-theoretic cryptosystems face obsolescence. Consequently, the National Institute of Standards and Technology (NIST) has selected lattice-based schemes such as CRYSTALS-Kyber as part of the post-quantum standardization process [2]. The fundamental security relies on the hardness in dealing with the Shortest Vector Problem (SVP): finding the non-zero vector with minimum Euclidean norm in a lattice $\mathcal{L}$ [10]. Solving SVP in high dimensions is notoriously difficult, this leads to the usage of lattice reduction algorithms. The algorithms such as LLL and BKZ transform an ill-conditioned basis into a nearly orthogonal one, significantly reducing the search space for exact solvers like Sieving or pure Enumeration [20]. The theoretical foundation of lattice-based cryptography is closely related to worst-case hardness results for lattice problems, beginning with Ajtai’s construction of hard lattice instances [1].

1.2 Motivation

The motivation is related to financial mathematics. From a financial-risk perspective, the transition to quantum-resistant cryptography is relevant because financial infrastructures especially the blockchain need to prepare for quantum-safe communication and long-term data protection [7]. Modern financial systems rely on cryptographic algorithms for secure communication, digital signatures, identity verification and transaction authentication. Post-quantum cryptography can therefore be regarded as part of long-term financial risk management. At the same time, DRL is widely connected with sequential decision-making problems in finance, such as portfolio re-balancing, optimal execution and cost-aware risk control. The BKZ scheduling problem has a similar structure: each reduction action has a computational cost. Its effect on the final basis quality may only become clear after several later reductions. This makes DRL a natural framework for studying adaptive lattice reduction from both an algorithmic and a decision-theoretic perspective.

Despite their widespread use, traditional lattice reduction algorithms still rely heavily on fixed reduction schedules. Standard BKZ usually follows a predetermined tour strategy and uses manually selected block sizes. Such a strategy is efficient and reliable in almost every conditions. However, it does not fully adapt to the geometric structure of each lattice instance. In high-dimensional lattices, some hidden relationships between basis vectors may remain difficult to remove after standard LLL or BKZ reduction. A fixed schedule may continue to spend computational resources on positions that bring little improvement. It may also fail to focus on blocks where the current basis still has strong non-orthogonal structure where requires deeper reduction.

This limitation motivates an adaptive scheduling method. In this essay, the BKZ reduction process is viewed as a sequential decision-making problem. At each step, the algorithm must decide the block size and the starting position to do local reduction. A larger block size usually produce both a stronger reduction and a higher computational cost. A reduction at one position may not immediately shorten the first vector. However, it may still improve the global basis profile and prepare the basis for future reductions. This delayed effect makes the problem suitable for Deep Reinforcement Learning (DRL).

Recent work has explored the use of DRL for BKZ control. Cheong et al. investigated DRL-based BKZ reduction and showed that learning-based control is possible [4]. Their results also indicate that the performance of a standard DRL agent can become unstable when the dimension increases. This suggests that the main difficulty is not only the choice of reinforcement learning algorithm. The state representation, neural architecture, action space and reward design are all important. A raw lattice basis matrix contains complete information, but its integer elements can be extremely large. Directly feeding it into a neural network may cause numerical instability and inefficient learning. Therefore, this thesis uses a geometric reparameterization. The state dimension is not significantly reduced. Instead, the integer coordinates are replaced by a safer representation that consists of angle correlations, length information, global orthogonality indicators and several scalar features.

1.3 Core Contributions

This thesis introduces a structured DRL framework for optimizing local BKZ reduction. The first contribution is to formulate local BKZ scheduling as a finite-horizon decision process. The agent observes the current geometric state of the basis and selects a local BKZ action. This formulation allows the reduction schedule to be learned adaptively rather than fixed manually. The second contribution is the state reparameterization. This method does not compress the state dimension. The raw basis $\mathbf{B} \in \mathbb{Z}^{n \times n}$ is replaced by a representation with comparable dimensionality. The implemented state consists of a cosine matrix of size approximately $n(n-1)$, a length profile of size n and a small number of scalar indicators. These components are concatenated into the state vector used by the neural network. The purpose of this transformation is to avoid the direct use of huge integer elements and to expose the geometric information that is more relevant to BKZ scheduling. The cosine matrix captures pairwise angular correlations. The length profile captures the Gram-Schmidt shape of the basis. The orthogonality defect and maximum cosine value provide global and worst-case local indicators.

The use of an Axial Convolutional Double DQN architecture for the cosine-matrix state is the third contribution. The cosine matrix has a basis-index structure, so row-wise and column-wise patterns are meaningful for local BKZ scheduling. The Axial CNN is used as a structure-preserving feature extractor, while Double DQN and noisy exploration are used for discrete action-value learning.

The fourth contribution is the detailed reward design that combines shortest-vector improvement, orthogonality improvement and computational cost. This reward gives the agent intermediate feedback after each local BKZ operation, reducing the difficulty caused by sparse terminal rewards.

The last contribution is a multi-language implementation that combines high-precision lattice reduction with neural-network training. The backend handles integer lattice operations, while PyTorch trains the DRL agent on geometric features extracted from the current basis. As a result, each language is capable of doing what it does best.

1.4 Notations

The main mathematical symbols used in this essay are summarized in Table 1.1 below.

Table 1.1: Mathematical Notations

Symbol	Description	Formula / Note
$\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_n]$	A lattice basis matrix. Each $\mathbf{b}_i$ is a basis vector.	$\mathbf{B} \in \mathbb{Z}^{n \times n}$
$\mathcal{L}(\mathbf{B})$	The lattice generated by the basis $\mathbf{B}$.	$\mathcal{L}(\mathbf{B}) = \{\sum_{i=1}^n z_i \mathbf{b}_i \mid z_i \in \mathbb{Z}\}$
$\mathbf{B}_t$	The current lattice basis at step t in one episode.	Updated by local BKZ and LLL
$\mathbf{b}_i^*$	The Gram-Schmidt orthogonalized vector corresponding to $\mathbf{b}_i$.	Used in the length profile
μ_{ij}	Gram-Schmidt coefficient.	$\mu_{ij} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\ \mathbf{b}_j^*\ ^2}$
$\lambda_1(\mathcal{L})$	The real shortest vector of the lattice.	$\lambda_1(\mathcal{L}) = \min_{v \in \mathcal{L} \setminus \{0\}} \ v\ $
$GH(\mathcal{L})$	Gaussian Heuristic of the lattice. An approximate measure of the shortest vector.	$GH(\mathcal{L}) \approx \sqrt{\frac{n}{2\pi e}} \det(\mathcal{L})^{1/n}$
$\rho(\mathbf{B})$	Norm-to-Gaussian-Heuristic ratio. It is the main short-vector quality metric.	$\rho(\mathbf{B}) = \frac{\ \mathbf{b}_1\ }{GH(\mathcal{L})}$
$\delta(\mathbf{B})$	Orthogonality defect of the basis.	$\delta(\mathbf{B}) = \frac{\prod_{i=1}^n \ \mathbf{b}_i\ }{ \det(\mathbf{B}) }$
$\delta^*(\mathbf{B})$	Normalized orthogonality defect.	$\delta^*(\mathbf{B}) = \delta(\mathbf{B})^{1/n}$
$D(\mathbf{B})$	Logarithmic orthogonality defect.	$D(\mathbf{B}) = \log \delta(\mathbf{B})$
$\mathbf{C}(\mathbf{B})$	Cosine matrix of pairwise basis-vector correlations.	$\mathbf{C}_{ij}(\mathbf{B}) = \left \frac{\mathbf{b}_i \cdot \mathbf{b}_j}{\ \mathbf{b}_i\ \ \mathbf{b}_j\ } \right $
$\mathbf{C}_{\max}(\mathbf{B})$	The maximum absolute cosine value, which measures the worst angle between vectors.	$\mathbf{C}_{\max}(\mathbf{B}) = \max_{i < j} \mathbf{C}_{ij}(\mathbf{B})$
$g(\mathbf{B})$	The normalized Gram-Schmidt length profile.	Constructed from $\log \ \mathbf{b}_i^*\ $
$\mathbf{a}_t$	The action selected by the DRL agent at step t .	$\mathbf{a}_t = (\beta_t, pos_t)$
β_t	BKZ block size selected at step t .	Controls local reduction strength
pos_t	Starting position of the local BKZ block.	The reduced block starts at pos_t

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the related literature and theoretical background. It first introduces classical lattice reduction algorithms, including LLL, BKZ and practical improvements of BKZ. Next, it discusses basis-quality indicators such as the

Gaussian Heuristic (GH), global orthogonality defect, Gram-Schmidt profile and cosine matrix. The chapter also reviews previous AI-based lattice reduction attempts and the DRL techniques that motivate this work, including Double DQN, noisy exploration and prioritized replay.

Chapter 3 illustrates the proposed methodology. It formulates local BKZ scheduling as a finite-horizon decision making problem and defines the state representation, action space, transition rule and reward function. This chapter also explains the geometric reparameterization of the raw lattice basis, the use of logarithmic quantities, the Gram-Schmidt length profile and the AxialCNN-DDQN architecture used to select local BKZ actions.

Chapter 4 describes the implementation and experimental setup. It explains how the C++/fpLLL backend, Rust feature-extraction layer and PyTorch reinforcement learning model are connected. It also describes the numerical scaling strategy for large integer bases, the construction of training instances, the stopping criteria, the baseline algorithms and the evaluation metrics used in the experiments.

Chapter 5 presents the experimental results and brief analysis. It reports the training dynamics, compares the initial LLL-preprocessed bases with the final DRL-BKZ results, and analyzes the results using the Norm/GH ratio, logarithmic orthogonality defect, maximum cosine value and representative cosine-matrix heatmaps. Incomplete baselines, ablation experiments and runtime profiling are discussed as limitations and future work.

Chapter 6 discusses the limitations and possible extensions of the proposed framework. It considers the computational cost of high-precision lattice reduction, the limitation of training on fixed dimensions, the lack of complete large-scale ablation experiments and possible future directions .

Finally, Chapter 7 concludes the thesis by summarizing the main findings and contributions.

Chapter 2

Literature Review

2.1 Classical Lattice Reduction: LLL, BKZ and Practical Improvements

A broader survey of lattice-based cryptography and its development can be found in Peikert[13]’s review. Lattice reduction is a fundamental tool for studying and solving SVP-related problems. Given a lattice basis ($\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_n]$) the purpose of reduction is to transform it into another basis of the same lattice with shorter and more orthogonal vectors. This transformation does not change the lattice itself. It only changes the representation of the lattice. A better representation can make later enumeration, sieving, or cryptanalytic procedures more efficient.

The LLL algorithm, introduced by Lenstra, Lenstra and Lovász, is one of the most important classical lattice reduction algorithms [9]. It runs in polynomial time and provides a reduced basis with provable approximation guarantees. LLL is efficient and stable as a preprocessing method. However, its reduction strength is limited. In high-dimensional SVP Challenge instances or cryptographic parameter settings, LLL alone is usually not sufficient to produce a basis whose first vector is close to the Gaussian Heuristic.

The BKZ algorithm, introduced by Schnorr and Euchner[15], improves reduction quality by applying stronger reduction to local blocks of basis vectors. Instead of only reducing adjacent vector pairs as in LLL, BKZ considers a block of size β and solves or approximates an SVP problem in the projected local sublattice. The block size β controls the trade-off between reduction quality and computational cost. A larger β usually gives a stronger reduction, but the computational cost also increases rapidly. BKZ 2.0 is employed in practical, which provides improved lattice-security estimates and incorporates more practical reduction techniques[3].

The practical behavior of BKZ has been studied extensively. Gama and Nguyen analyzed the prediction of lattice reduction quality and connected basis quality with indicators such as the root Hermite factor and the Gram-Schmidt profile [6]. More recent improvements focus not only on choosing a large block size, but also on deciding where and when expensive local reductions should be applied. Zhao and Ding proposed several practical improvements to BKZ, including local basis processing, jumping over several indices, reducing only the necessary part of the basis in later tours and applying a heavier final SVP subroutine [19]. These techniques suggest an important idea: not all local reductions are equally valuable and the order and location of

reductions can significantly affect both cost and output quality.

This observation is closely related to the motivation of this thesis. Classical BKZ and its improved variants are mathematically reliable, but the scheduling rules are still mainly designed by human heuristics. A fixed or manually designed schedule may not fully adapt to the geometry of each individual lattice basis. This motivates the study of a learning-based scheduler that can decide local BKZ actions according to the current state of the basis.

2.2 Basis Quality, Gaussian Heuristic and Orthogonality

Lattices have two roles in cryptology: they can be used to construct cryptographic primitives and also to attack poorly parameterized systems[12]. Furthermore, the quality of a lattice basis can be measured from several perspectives. The most direct indicator for SVP-related reduction is the length of the first basis vector. In this thesis, the main short-vector quality metric is the Norm/GH ratio:

$$\rho(\mathbf{B}) = \frac{\|\mathbf{b}_1\|}{GH(\mathcal{L})}.$$

Here $GH(\mathcal{L})$ denotes the Gaussian Heuristic:

$$GH(\mathcal{L}) \approx \sqrt{\frac{n}{2\pi e}} \det(\mathcal{L})^{1/n}.$$

A smaller value of $\rho(\mathbf{B})$ indicates that the first vector is closer to the expected shortest-vector scale of the lattice. In the SVP Challenge setting, a ratio close to or below 1.05 is often used as a practical target for evaluating short-vector quality. Therefore, this thesis uses $\rho(\mathbf{B}) \leq 1.05$ as an empirical success threshold in the experiments.

However, the first-vector length is not the only useful indicator. Orthogonality also plays an important role in lattice reduction. The orthogonality defect is defined as

$$\delta(\mathbf{B}) = \frac{\prod_{i=1}^n \|\mathbf{b}_i\|}{|\det(\mathbf{B})|}.$$

By Hadamard's inequality, $\delta(\mathbf{B}) \geq 1$ and equality holds when the basis vectors are mutually orthogonal. Therefore, the defect measures how far the basis is from an orthogonal basis. In numerical implementation, it is often more stable to use the logarithmic form:

$$D(\mathbf{B}) = \log \delta(\mathbf{B}) = \sum_{i=1}^n \log \|\mathbf{b}_i\| - \log \det(\mathcal{L}).$$

The logarithmic form avoids directly computing products of many vector norms, which can become extremely large in high dimensions.

Another important local indicator is the cosine matrix:

$$\mathbf{C}_{ij}(\mathbf{B}) = \left| \frac{\mathbf{b}_i \cdot \mathbf{b}_j}{\|\mathbf{b}_i\| \|\mathbf{b}_j\|} \right|.$$

The value $\mathbf{C}_{ij}(\mathbf{B})$ measures the absolute angular correlation between two basis vectors. A value close to zero indicates near orthogonality. A value close to one indicates strong correlation. The maximum cosine

$$\mathbf{C}_{\max}(\mathbf{B}) = \max_{i < j} \mathbf{C}_{ij}(\mathbf{B})$$

captures the worst local pairwise correlation in the current basis.

These indicators are important because basis quality affects downstream SVP-solving procedures. In particular, sieving and enumeration can benefit from a well-reduced input basis. Zhao, Ding and Yang showed that the efficiency of sieving with streaming memory access is closely related to basis quality [20]. Therefore, this thesis evaluates not only the first-vector quality, but also the global and local orthogonality structure of the reduced basis. For high-dimensional SVP, sieving algorithms' performance are also affected by the quality of the reduced basis[8].

2.3 AI-Based Lattice Reduction and Its Difficulties

Recent research has explored the use of machine learning and reinforcement learning for lattice reduction. Cheong et al. investigated DRL-based control of BKZ and showed that learning-based reduction policies are possible [4]. Their work is important because it demonstrates that BKZ scheduling can be treated as a sequential decision-making problem rather than only as a fixed algorithmic routine.

At the same time, existing AI-based reduction methods also reveal several difficulties. First, directly using the raw lattice basis B as neural-network input can be numerically unstable. The entries of SVP Challenge bases can be very large and their scale can vary significantly across dimensions and random seeds. A neural network trained on raw coordinates must learn numerical normalization and geometric decision-making at the same time. This can make training inefficient or unstable.

Second, reward sparsity is a serious issue. If the agent receives meaningful feedback only after a full tour or at the end of an episode, it becomes difficult to assign credit to individual BKZ actions. A local reduction may not immediately shorten b_1 , but it may improve the Gram-Schmidt profile and make later reductions more effective. Therefore, a dense reward based on intermediate geometric improvement is useful.

These observations motivate a more stable geometric representation rather than a direct coordinate-based representation. Angle correlations, Gram-Schmidt length information and orthogonality-related indicators can describe the basis in a way that is more suitable for neural learning. The precise construction of the state representation used in this thesis is given in Chapter 3.

2.4 DRL Architectures for Adaptive BKZ Scheduling

Deep Reinforcement Learning is suitable for problems where an agent makes sequential decisions and receives delayed rewards [17]. Deep Q-Networks showed that deep neural networks can approximate action-value functions for high-dimensional sequential decision problems[11]. This structure naturally appears in BKZ scheduling. At each step, the agent chooses a local reduction action, such as a block size and a starting position. The action has an immediate computational cost, but its effect on the final basis quality may only become clear after several later reductions. This is also related to cost-aware sequential decision problems in financial mathematics, where short-term cost and long-term performance must be balanced.

Since the action space in this thesis is discrete, a value-based DRL framework is appropriate. Standard DQN may suffer from overestimation bias when selecting actions. Double DQN reduces

this problem by separating action selection from target evaluation [18]. This is useful for BKZ scheduling because many local actions may have similar values and overestimating a few actions may lead to unstable policies.

Exploration and sample efficiency are also important. Simple ϵ -greedy exploration may waste many trials on ineffective or expensive local reductions. Noisy Networks provide a more structured exploration mechanism by adding learnable parameter noise to the network [5]. Prioritized Experience Replay further improves learning efficiency by replaying transitions with larger temporal-difference errors more frequently [14]. In the BKZ setting, such transitions may correspond to rare but important reductions that significantly improve the first vector length, the orthogonality defect, or the maximum cosine value.

The state representation also requires a suitable neural architecture. A fully connected network would flatten the cosine matrix and ignore much of its geometric structure. However, the rows and columns of the cosine matrix are both indexed by basis vectors and local BKZ actions also operate along these indices. Therefore, a structure-preserving feature extractor is desirable. This motivates the use of an Axial CNN, which can model row-wise and column-wise dependencies in the cosine matrix before the extracted features are used for action-value estimation. The detailed network architecture is presented in Chapter 3.

2.5 Research Gap

The existing literature provides powerful classical algorithms for lattice reduction and also shows the potential of learning-based BKZ control. Classical BKZ methods are reliable and effective, but their schedules are usually fixed or manually designed. Practical BKZ improvements show that local processing choices, jumping strategies and final heavy reductions can affect efficiency, but these choices still depend on human-designed heuristics [19]. Existing DRL approaches show that learning is possible, but they can suffer from coordinate instability, sparse rewards and insufficient geometric representation [4].

The gap is therefore not the absence of strong lattice reduction algorithms. The gap is the lack of an adaptive scheduling mechanism that can use stable geometric information to decide which local reductions are worth applying. This thesis addresses this gap by combining BKZ 2.0 with a DRL scheduler. The proposed approach keeps the mathematical reliability of classical lattice reduction while learning how to schedule local reductions more adaptively.

Chapter 3

Methodology

This chapter presents the methodology of the DRL-controlled BKZ framework. The chapter is divided into four main parts. Section 3.1 gives a high-level overview of the method. Section 3.2 introduces the mathematical quantities used by the agent and proves the main properties of the state representation. Section 3.3 describes the technical design of the agent, including the MDP formulation, state construction, action space, reward function, AxialCNN-DDQN-Noisy architecture and training protocol. Section 3.4 summarizes the chapter.

3.1 Overview of the DRL-Controlled BKZ Framework

The purpose of this method is to learn an adaptive schedule for local BKZ reduction. Given a lattice basis

$$\mathbf{B}_t = [\mathbf{b}_1^{(t)}, \dots, \mathbf{b}_n^{(t)}] \in \mathbb{Z}^{n \times n}$$

the agent observes a geometric state vector $\mathbf{s}_t$, selects a local BKZ action

$$\mathbf{a}_t = (\beta_t, pos_t)$$

and receives a reward based on the change in basis quality after the reduction.

The framework does not directly feed the raw integer basis $\mathbf{B}_t$ into the neural network. Instead, the basis is reparameterized into a numerically stable geometric state:

$$\mathbf{s}_t = [\text{vec}(\mathbf{T}(\mathbf{C}_t)), \mathbf{g}_t, \mathbf{u}_t]$$

Here $\mathbf{C}_t$ is the cosine matrix at step t , $\mathbf{T}(\mathbf{C}_t)$ is the spliced cosine-token matrix used by the CNN branch, $\mathbf{g}_t$ is the normalized Gram-Schmidt length profile and $\mathbf{u}_t$ is a vector of five scalar indicators.

The method can be summarized as:

$$\mathbf{B}_t \longrightarrow \text{Geometric} \longrightarrow \mathbf{s}_t \longrightarrow \text{AxialCNN-DDQN} \longrightarrow \mathbf{a}_t \longrightarrow \text{Local BKZ 2.0} \longrightarrow \mathbf{B}_{t+1}.$$

The agent is trained on multiple SVP Challenge seeds of the same dimension. In the current setting, the seed set is

$$\mathcal{I} = \{0, 1, 2, 3, 5, 7, 9, 11, 13\}.$$

At the beginning of each episode, the environment randomly selects one seed from $\mathcal{I}$ and resets the basis to the corresponding initial lattice instance. This prevents the agent from overfitting to a single initial basis. Training stops when at least 6/9 seeds reach the target

$$\rho(\mathbf{B}) = \frac{\|\mathbf{b}_1\|}{GH(\mathcal{L})} \leq 1.05$$

or when the maximum training episode is reached.

The threshold 1.05 is the main optimization target of this thesis while orthogonality improvement is used as an auxiliary objective. Orthogonality defect helps prepare a better basis profile and stabilize the reduction process, but the primary target is still to obtain a sufficiently short first vector ($\mathbf{b}_1$).

Algorithm 1: DRL-Controlled Local BKZ

Require: Raw seed set $\mathcal{I} = \{0, 1, 2, 3, 5, 7, 9, 11, 13\}$, action set $\mathcal{A}$, maximum episode length T

```

1: Initialize replay memory and AxialCNN-DDQN parameters
2: Initialize an empty cached basis pool  $\mathcal{P}_{\text{LLL}}$ 
3: for each seed  $i \in \mathcal{I}$  do
4:   Load raw initial basis  $\mathbf{B}_{\text{raw}}^{(i)}$ 
5:   Apply LLL preprocessing once:  $\tilde{\mathbf{B}}_0^{(i)} \leftarrow \text{LLL}(\mathbf{B}_{\text{raw}}^{(i)})$ 
6:   Store  $\tilde{\mathbf{B}}_0^{(i)}$  in cached pool  $\mathcal{P}_{\text{LLL}}$ 
7: end for
8: while fewer than 6 seeds satisfy  $\rho(\mathbf{B}) \leq 1.05$  do
9:   Randomly sample a seed  $i \in \mathcal{I}$ 
10:  Reset environment by cloning cached basis  $\tilde{\mathbf{B}}_0^{(i)}$  from  $\mathcal{P}_{\text{LLL}}$ 
11:  Set current basis  $\mathbf{B}_0 \leftarrow \tilde{\mathbf{B}}_0^{(i)}$ 
12:  for  $t = 0, 1, \dots, T-1$  do
13:    Construct state  $\mathbf{s}_t \leftarrow f(\mathbf{B}_t)$ 
14:    Select local BKZ action  $\mathbf{a}_t = (\beta_t, \text{pos}_t)$ 
15:    Apply Local BKZ 2.0:  $\hat{\mathbf{B}}_t \leftarrow \text{LocalBKZ}_{2.0}(\mathbf{B}_t, \beta_t, \text{pos}_t)$ 
16:    if periodic LLL condition is satisfied, e.g. every 3 steps then
17:      Apply LLL stabilization:  $\mathbf{B}_{t+1} \leftarrow \text{LLL}(\hat{\mathbf{B}}_t)$ 
18:    else
19:      Set  $\mathbf{B}_{t+1} \leftarrow \hat{\mathbf{B}}_t$ 
20:    end if
21:    Construct next state  $\mathbf{s}_{t+1} \leftarrow f(\mathbf{B}_{t+1})$ 
22:    Compute reward  $r_t$ 
23:    Store transition  $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$ 
24:    Update DDQN parameters using replay samples
25:  end for
26:  Apply final large-block BKZ polish:  $\mathbf{B}_T^{\text{final}} \leftarrow \text{LLL}(\text{LocalBKZ}_{2.0}(\mathbf{B}_T, \beta_{\text{final}}, 0))$ 
27:  Update best ratio:  $\rho_{\text{best}}^{(i)} \leftarrow \min(\rho_{\text{best}}^{(i)}, \rho(\mathbf{B}_T^{\text{final}}))$ 
28: end while
29: return Trained agent parameters and training log

```

3.2 Mathematical Quantities and State Representation

This section introduces the mathematical quantities used in the state representation. These quantities are independent of the neural-network agent. They describe the geometry of the lattice basis and provide the agent with stable inputs.

Norm/GH Ratio

The primary short-vector quality metric is

$$\rho(\mathbf{B}) = \frac{\|\mathbf{b}_1\|}{GH(\mathcal{L})}$$

The Gaussian Heuristic is approximated by

$$GH(\mathcal{L}) \approx \sqrt{\frac{n}{2\pi e}} \det(\mathcal{L})^{1/n}$$

A smaller $\rho(\mathbf{B})$ means that the first basis vector is closer to the expected shortest-vector scale. The main empirical target in this thesis is

$$\rho(\mathbf{B}) \leq 1.05$$

For numerical stability, the implementation uses

$$\log \rho(\mathbf{B}) = \log \|\mathbf{b}_1\| - \log GH(\mathcal{L})$$

The displayed ratio is recovered by

$$\rho(\mathbf{B}) = \exp(\log \rho(\mathbf{B}))$$

Cosine Matrix

The cosine matrix records pairwise angular correlations between basis vectors in $\mathbb{R}^{(n-1) \times (n-1)}$:

$$\mathbf{C}_{ij}(\mathbf{B}) = \left| \frac{\mathbf{b}_i \cdot \mathbf{b}_j}{\|\mathbf{b}_i\| \|\mathbf{b}_j\|} \right|, \quad i < j.$$

The absolute value is used because both strongly positive and strongly negative correlations indicate non-orthogonality. The sign is less important than the magnitude for the purpose of selecting local blocks to reduce.

A value close to zero means that two vectors are nearly orthogonal. A value close to one means that two vectors are strongly correlated. Therefore, the cosine matrix provides local geometric information that a single global index cannot capture and natively adapted to CNN.

Cosine-Token Matrix Used by CNN

The cosine matrix $\mathbf{C}(\mathbf{B})$ is asymmetric: A lower triangular matrix. We need a rectangular matrix so that it can better fit the CNN network.

As a result, the cosine-token matrix is denoted by

$$\mathbf{C} + \mathbf{C}^T = \mathbf{T}(\mathbf{C}) \in \mathbb{R}^{(n-1) \times n}$$

It should be understood as a structured token representation of the off-diagonal cosine information, rather than the original symmetric cosine matrix itself. The purpose of $\mathbf{T}(\mathbf{C})$ is to preserve pairwise angular information in a form that can be processed by convolutional layers.

The cosine-token matrix is flattened into an 1-D vector before being concatenated into the full state:

$$\text{vec}(\mathbf{T}(\mathbf{C})) \in \mathbb{R}^{(n-1)n}$$

The full state vector is

$$\mathbf{s}_t = [\text{vec}(\mathbf{T}(\mathbf{C}_t)), \mathbf{g}_t, \mathbf{u}_t]$$

where $\mathbf{g}_t \in \mathbb{R}^n$ is the normalized Gram-Schmidt length profile and $\mathbf{u}_t \in \mathbb{R}^5$ is the scalar feature vector. Thus, the total state dimension is

$$(n-1)n + n + 5 = n^2 + 5$$

Inside the neural network, the first $(n - 1)n$ entries of $\mathbf{s}_i$ are separated and reshaped back into

$$\mathbf{X}_C \in \mathbb{R}^{N_{\text{batch}} \times 1 \times (n-1) \times n}$$

The channel dimension is one because the cosine-token matrix is treated as a single-channel structured input. The Axial CNN then applies row-direction and column-direction convolutions to $\mathbf{X}_C$, so that both views of the pairwise-correlation structure can contribute to the action-value estimation.

Gram-Schmidt Length Profile

The implementation uses the logarithmic Gram-Schmidt length vector, showing the co-length information of all vectors.

$$\ell(\mathbf{B}) = [\log \|\mathbf{b}_1^*\|, \dots, \log \|\mathbf{b}_n^*\|]$$

To make this vector numerically stable, each entry is measured relative to the Gaussian Heuristic. Let

$$\log \rho(\mathbf{B}) = \log \|\mathbf{b}_1^*\| - \log GH(\mathcal{L})$$

be the initial log norm-to-Gaussian-Heuristic ratio after LLL preprocessing. The normalization scale is defined as

$$s_\rho = \max(|\log \rho(\mathbf{B})|, 1)$$

This scale corresponds to the implementation variable `ratio_scale`. It is used as a numerical ruler for log-ratio-related features.

The normalized Gram-Schmidt profile is then defined by

$$\mathbf{g}_i(\mathbf{B}) = \tanh\left(\frac{\log \|\mathbf{b}_i^*\| - \log GH(\mathcal{L})}{s_\rho}\right)$$

The subtraction of $\log GH(\mathcal{L})$ removes the global lattice scale, while division by s_ρ and the $\tanh$ function keep the input in a stable range for neural-network training. The total number of parameters combining the length profile with the cosine matrix is $n \times n$ and all elements are within the range of $[-1, 1]$. Knowing these two can restore the entire matrix, which is the almost lossless reparameterization of the original matrix. Since replacing a basis vector b_i by $-b_i$ does not change the generated lattice (symmetric property), the sign of the pairwise inner product is not essential for the local BKZ scheduling task.

Orthogonality Defect

The implementation uses the logarithmic orthogonality defect

$$D(\mathbf{B}) = \log \delta(\mathbf{B}) = \sum_{i=1}^n \log \|\mathbf{b}_i\| - \log \det(\mathcal{L})$$

This value is a scalar. It is not divided by n in the implementation. To make it suitable for neural-network input, it is normalized by a defect scale s_D . Let

$$s_D = \max(|D(\mathbf{B})|, 1)$$

where $\mathbf{B}_0$ is the LLL-preprocessed initial basis. The normalized defect feature is then

$$\tilde{D}(\mathbf{B}) = \tanh\left(\frac{D(\mathbf{B})}{s_D}\right)$$

Here s_D corresponds to the implementation variable `defect_scale`. It serves only as a numerical normalization factor and is not a new lattice invariant.

Scalar Indicators

The scalar part of the state is

$$\mathbf{u}_t = [C_{\max}(\mathbf{B}_t), \tilde{D}(\mathbf{B}_t), \widetilde{\log \rho}(\mathbf{B}_t), S_g(\mathbf{B}_t), t/T]$$

These five scalar indicators provide a compact summary of the global condition of the current basis. They are defined as follows.

1. **Maximum cosine.** The first scalar is the maximum absolute cosine value:

$$C_{\max}(\mathbf{B}) = \max_{i < j} \mathbf{C}_{ij}(\mathbf{B})$$

It measures the worst pairwise correlation in the current basis. A larger value means that at least one pair of basis vectors remains strongly non-orthogonal.

2. **Scaled logarithmic orthogonality defect.** The second scalar is the scaled logarithmic defect:

$$\tilde{D}(\mathbf{B}) = \tanh\left(\frac{D(\mathbf{B})}{s_D}\right)$$

Here $D(\mathbf{B}) = \log \delta(\mathbf{B})$ is the unnormalized logarithmic orthogonality defect and s_D is the defect normalization scale. This feature gives the agent a global measure of basis orthogonality.

3. **Scaled log Norm/GH ratio.** The third scalar is

$$\widetilde{\log \rho}(\mathbf{B}) = \tanh\left(\frac{\log \rho(\mathbf{B})}{s_\rho}\right)$$

It measures the quality of the first vector relative to the Gaussian Heuristic after numerical normalization. This scalar is directly related to the main target

$$\rho(\mathbf{B}) \leq 1.05$$

4. **Global Gram-Schmidt slope indicator.** The fourth scalar is

$$S_g(\mathbf{B}) = \tanh\left(\frac{\log \|\mathbf{b}_1^*\| - \log \|\mathbf{b}_n^*\|}{s_\rho}\right)$$

This is not a local derivative. It is the front-back difference of the Gram-Schmidt profile. It tells the agent whether the basis profile is steep or relatively flat.

5. **Normalized step progress.** The fifth scalar is

$$\frac{t}{T}$$

It informs the agent of the remaining decision budget in the current episode.

Why the Raw Basis Is Not Used

The raw basis matrix $\mathbf{B}$ contains complete information about the lattice, but its entries can be extremely large. Directly feeding $\mathbf{B}$ into a neural network can cause unstable training and possible gradient explosion. It also forces the network to learn numerical scaling and geometric decision-making at the same time.

The proposed state representation is not a compression. The raw basis has n^2 entries, while the implemented state has dimension

$$(n-1)n + n + 5 = n^2 + 5$$

Therefore, the change is a reparameterization rather than a dimensional reduction. The purpose is to replace unstable integer coordinates with stable geometric quantities.

3.3 Mathematical Properties of the State Representation

This section provides the mathematical justification for the geometric state representation used in this thesis. The purpose is not to prove that the representation fully reconstructs the original basis. Instead, the goal is to show that the selected features are meaningful, numerically stable and suitable for local BKZ scheduling.

The main state representation is

$$\mathbf{s}_t = [\text{vec}(\mathbf{T}(\mathbf{C}_t)), \mathbf{g}_t, \mathbf{u}_t]$$

where $\mathbf{T}(\mathbf{C}_t)$ is the cosine-token matrix, $\mathbf{g}_t$ is the normalized Gram-Schmidt length profile and $\mathbf{u}_t$ is the scalar feature vector. The proofs below mainly justify the orthogonality defect, the scale invariance of the cosine matrix and the scale invariance of the normalized Gram-Schmidt profile.

Orthogonality Defect and Hadamard Inequality

Lemma 3.1 (Lower Bound of Orthogonality Defect) *For any full-rank basis*

$$\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_n]$$

the orthogonality defect satisfies

$$\delta(\mathbf{B}) \geq 1$$

Equivalently, if taking logarithms:

$$D(\mathbf{B}) = \log \delta(\mathbf{B}) \geq 0$$

Proof. By Hadamard's inequality,

$$|\det(\mathbf{B})| \leq \prod_{i=1}^n \|\mathbf{b}_i\|$$

Therefore,

$$\delta(\mathbf{B}) = \frac{\prod_{i=1}^n \|\mathbf{b}_i\|}{|\det(\mathbf{B})|} \geq 1$$

Taking logarithms gives

$$D(\mathbf{B}) = \log \delta(\mathbf{B}) \geq 0$$

Equality holds when the basis vectors are mutually orthogonal. $\square$

This lemma justifies the use of $D(\mathbf{B})$ as a global orthogonality indicator. A smaller $D(\mathbf{B})$ means that the basis is closer to being orthogonal. In the implementation, $D(\mathbf{B})$ is used in its logarithmic form because it avoids the direct product $\prod_i \|\mathbf{b}_i\|$, which can become extremely large.

Lemma 3.2 (Scale Invariance of the Cosine Matrix) *Let*

$$\mathbf{B}' = \alpha \mathbf{B}$$

for $\alpha > 0$. Then

$$\mathbf{C}_{ij}(\mathbf{B}') = \mathbf{C}_{ij}(\mathbf{B})$$

Proof. By definition,

$$\mathbf{C}_{ij}(\mathbf{B}') = \left| \frac{(\alpha \mathbf{b}_i) \cdot (\alpha \mathbf{b}_j)}{\|\alpha \mathbf{b}_i\| \|\alpha \mathbf{b}_j\|} \right|$$

Since

$$(\alpha \mathbf{b}_i) \cdot (\alpha \mathbf{b}_j) = \alpha^2 (\mathbf{b}_i \cdot \mathbf{b}_j)$$

and

$$\|\alpha \mathbf{b}_i\| \|\alpha \mathbf{b}_j\| = \alpha^2 \|\mathbf{b}_i\| \|\mathbf{b}_j\|$$

we obtain

$$\mathbf{C}_{ij}(\mathbf{B}') = \left| \frac{\alpha^2 (\mathbf{b}_i \cdot \mathbf{b}_j)}{\alpha^2 \|\mathbf{b}_i\| \|\mathbf{b}_j\|} \right| = \mathbf{C}_{ij}(\mathbf{B})$$

$\square$

This proposition explains why the cosine matrix is more stable than the raw basis coordinates. Even if all integer entries of the basis are scaled, the angular correlation structure does not change. This is one reason why the cosine matrix is suitable for neural-network input.

Lemma 3.3 (Scale Invariance of the Normalized Gram-Schmidt Profile) *Let*

$$\ell(\mathbf{B}) = [\log \|\mathbf{b}_1^*\|, \dots, \log \|\mathbf{b}_n^*\|]$$

be the logarithmic Gram-Schmidt length vector. To make this vector comparable across different basis scales, each entry is normalized relative to the Gaussian Heuristic:

$$g_i(\mathbf{B}) = \tanh \left(\frac{\log \|\mathbf{b}_i^*\| - \log GH(\mathcal{L}(\mathbf{B}))}{s_\rho} \right)$$

Here s_ρ is the ratio normalization scale. In the implementation, it corresponds to `ratio_scale`. It is initialized from the LLL-preprocessed basis:

$$s_\rho = \max(|\log \rho(\mathbf{B}_0)|, 1)$$

It is not a new lattice invariant. It is only a numerical scale used to keep log-ratio-related features in a stable range.

Lemma 3.4 (Scale Invariance of the Normalized Gram-Schmidt Profile) *Let*

$$\mathbf{B}' = \alpha \mathbf{B}$$

for $\alpha > 0$. If the same normalization scale s_ρ is used, then

$$g_i(\mathbf{B}') = g_i(\mathbf{B})$$

Proof. Under the scaling $\mathbf{B}' = \alpha \mathbf{B}$, the Gram-Schmidt vectors scale as

$$(\mathbf{b}'_i)^* = \alpha \mathbf{b}_i^*$$

Thus,

$$\log \|(\mathbf{b}'_i)^*\| = \log \alpha + \log \|\mathbf{b}_i^*\|$$

The lattice determinant scales as

$$\det(\mathcal{L}(\mathbf{B}')) = \alpha^n \det(\mathcal{L}(\mathbf{B}))$$

Therefore,

$$GH(\mathcal{L}(\mathbf{B}')) = \alpha GH(\mathcal{L}(\mathbf{B}))$$

Taking logarithms gives

$$\log GH(\mathcal{L}(\mathbf{B}')) = \log \alpha + \log GH(\mathcal{L}(\mathbf{B}))$$

Hence,

$$\log \|(\mathbf{b}'_i)^*\| - \log GH(\mathcal{L}(\mathbf{B}')) = \log \|\mathbf{b}_i^*\| - \log GH(\mathcal{L}(\mathbf{B}))$$

Applying the same division by s_ρ and the same $\tanh$ function gives

$$g_i(\mathbf{B}') = g_i(\mathbf{B})$$

□

This proposition shows that the normalized Gram-Schmidt profile measures the shape of the basis relative to the lattice scale. It does not simply measure the absolute size of the entries.

Geometric Information and Limitations of Absolute Cosine

The cosine matrix and the Gram-Schmidt profile provide complementary information. The cosine matrix describes local pairwise angular correlations, while the Gram-Schmidt profile describes the projected length distribution of the basis. Together, they give the agent information about both local non-orthogonality and global basis shape.

The implementation uses the absolute cosine value

$$\mathbf{C}_{ij}(\mathbf{B}) = \left| \frac{\mathbf{b}_i \cdot \mathbf{b}_j}{\|\mathbf{b}_i\| \|\mathbf{b}_j\|} \right|$$

This means that the sign of the inner product is not retained. Therefore, the state does not fully reconstruct the signed Gram-Schmidt coefficients or the original basis. This is an intentional design choice. The scheduling task mainly requires the agent to detect the magnitude and location of strong non-orthogonal correlations. For this purpose, the absolute cosine value is sufficient and more numerically stable.

Feature Extraction Complexity

The original purpose of the complexity comparison is to show that the cosine matrix is more expensive than a scalar orthogonality index, but both are in the same leading complexity class. This justifies the use of cosine information because it provides richer local geometry while remaining computationally feasible.

Let

$$\mathbf{B} \in \mathbb{R}^{n \times n}$$

The cost of computing the orthogonality defect is dominated by determinant computation. Using Gaussian elimination or LU decomposition,

$$T_\delta \approx 2n^2 + \frac{2}{3}n^3 \approx \frac{2}{3}n^3$$

The $2n^2$ term comes from computing vector norms, while the $O(n^3)$ term comes from determinant computation.

For the cosine matrix, there are approximately

$$\frac{n(n-1)}{2}$$

off-diagonal vector pairs. Each dot product costs approximately $2n$ floating-point operations. Thus,

$$T_C \approx \frac{n^2}{2} \cdot 2n = n^3$$

The leading-order ratio is therefore

$$\frac{T_C}{T_\delta} \approx \frac{n^3}{\frac{2}{3}n^3} = 1.5$$

Thus, the cosine matrix is about 1.5 times as expensive as the scalar defect calculation in this simplified model, but both remain $O(n^3)$. The additional cost is justified because the cosine matrix provides local pairwise information that cannot be captured by a single global defect value.

3.4 Agent Design and Learning Procedure

This section describes the agent-specific part of the methodology. It includes the MDP formulation, the reward design, the AxialCNN-DDQN architecture and the training protocol.

MDP Formulation

The local BKZ scheduling process is formulated as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, T)$$

The observed state is

$$\mathbf{s}_t = f(\mathbf{B}_t),$$

where $\mathbf{B}_t$ is the current basis stored in the environment and $f(\cdot)$ is the geometric feature-extraction function. The action is

$$\mathbf{a}_t = (\beta_t, pos_t)$$

where β_t is the selected BKZ block size and pos_t is the starting position of the local block. After the action is selected, the environment first applies local BKZ 2.0 to the selected block:

$$\widehat{\mathbf{B}}_{t+1} = \text{LocalBKZ}_{2.0}(\mathbf{B}_t, \beta_t, pos_t)$$

Here $\widehat{\mathbf{B}}_{t+1}$ denotes the basis immediately after the local BKZ operation but before any possible global stabilization. Local BKZ changes the basis representation but preserves the underlying lattice, as in standard lattice reduction.

A full-basis LLL reduction is not applied after every local BKZ step. Instead, it is applied periodically after every three local BKZ operations. Thus, the environment transition is

$$\mathbf{B}_{t+1} = \begin{cases} \text{LLL}(\widehat{\mathbf{B}}_{t+1}), & \text{if } (t+1) \equiv 0 \pmod{3} \\ \widehat{\mathbf{B}}_{t+1}, & \text{otherwise} \end{cases}$$

The periodic LLL step is not treated as an action selected by the agent. It is part of the environment dynamics. Its purpose is to propagate local improvements to the whole basis. In practice, a local BKZ operation may improve only a selected block. A later full-basis LLL step can spread this improvement through size reduction and swaps. This gives two advantages: it improves the overall basis profile and it can help transfer useful improvements toward the front of the basis, especially toward $\mathbf{b}_1$.

At the terminal step, a large-block is applied at the beginning of the basis:

$$\mathbf{B}_T^{\text{final}} = \text{LLL}(\text{LocalBKZ}_{2.0}(\mathbf{B}_T, \beta_{\text{final}}, 0)).$$

The reason is that the final evaluation mainly depends on

$$\rho(\mathbf{B}) = \frac{\|\mathbf{b}_1\|}{GH(\mathcal{L})}.$$

The earlier learned actions improve the basis profile and orthogonality, while the final large block directly tries to improve $\mathbf{b}_1$.

Reward and Penalty Function

The reward function is designed around the main target

$$\rho(\mathbf{B}) \leq 1.05$$

Therefore, the first-vector reward has the largest conceptual priority. Orthogonality-related rewards are auxiliary. They are included because better orthogonality can help later BKZ steps and improve basis stability, but they are not the primary target.

The first reward component is

$$R_{\rho,t} = \log \|\mathbf{b}_1^{(t)}\| - \log \|\mathbf{b}_1^{(t+1)}\|$$

If $\mathbf{b}_1$ becomes shorter, then $R_{\rho,t} > 0$.

The maximum-cosine improvement is

$$R_{C,t} = C_{\max}(\mathbf{B}_t) - C_{\max}(\mathbf{B}_{t+1})$$

The defect improvement is

$$R_{D,t} = D(\mathbf{B}_t) - D(\mathbf{B}_{t+1})$$

The total reward is

$$r_t = w_\rho R_{\rho,t} + w_C R_{C,t} + w_D R_{D,t} - w_\beta \frac{\beta_t}{\beta_{\max}} + r_{\text{best}} - r_{\text{repeat}}$$

The weights are chosen according to the target hierarchy below:

$$w_\rho > w_D \ \& \ w_C$$

The term $w_\rho R_{\rho,t}$ emphasizes shortening $\mathbf{b}_1$. The terms $w_C R_{C,t}$ and $w_D R_{D,t}$ encourage local and global orthogonality improvement. The block-size penalty:

$$w_\beta \frac{\beta_t}{\beta_{\max}}$$

prevents the agent from always selecting the most expensive block size.

The bonus r_{best} rewards a new episode-level or global best ratio. The repeated-action penalty r_{repeat} discourages the agent from staying around a small set of actions that produce weak but nonzero rewards.

The reward contains history-related shaping terms, such as best-basis bonuses and repeated-action penalties. Therefore, the practical reward is not a strictly Markovian function of only

$$(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}).$$

This is an engineering choice to improve exploration and training stability. The main geometric reward components remain based on immediate basis-quality changes.

Algorithm 2 Reward Computation

Require: Old basis $\mathbf{B}_t$, new basis $\mathbf{B}_{t+1}$, action $\mathbf{a}_t = (\beta_t, pos_t)$, recent action window $\mathcal{W}_t$

- 1: Compute first-vector improvement:

$$R_{\rho,t} = \log \|\mathbf{b}_1^{(t)}\| - \log \|\mathbf{b}_1^{(t+1)}\|$$

- 2: Compute maximum-cosine improvement:

$$R_{C,t} = C_{\max}(\mathbf{B}_t) - C_{\max}(\mathbf{B}_{t+1})$$

- 3: Compute orthogonality-defect improvement:

$$R_{D,t} = D(\mathbf{B}_t) - D(\mathbf{B}_{t+1})$$

- 4: Compute block-size penalty:

$$r_{\beta,t} = w_{\beta} \frac{\beta_t}{\beta_{\max}}$$

- 5: Count how many times the current action appears with no improvements in the recent 8-step window:

$$k_t = \#\{\tau \in \mathcal{W}_t : \mathbf{a}_{\tau} = \mathbf{a}_t\}$$

- 6: Compute repeated-action penalty:

$$r_{\text{repeat},t} = c_{\text{rep}} k_t$$

- 7: Add best-basis bonus
- $r_{\text{best},t}$
- if a new episode-level or global best ratio is found

- 8: Return

$$r_t = w_{\rho} R_{\rho,t} + w_C R_{C,t} + w_D R_{D,t} - r_{\beta,t} + r_{\text{best},t} - r_{\text{repeat},t}$$

AxialCNN-DDQN Architecture

The network receives

$$\mathbf{s}_t = [\text{vec}(\mathbf{T}(\mathbf{C}_t)), \mathbf{g}_t, \mathbf{u}_t]$$

The architecture has two parallel branches.

The first branch is the cosine branch. It takes

$$\text{vec}(\mathbf{T}(\mathbf{C}_t))$$

and reshapes it into the CNN input tensor

$$\mathbf{X}_C \in \mathbb{R}^{N_{\text{batch}} \times 1 \times (n-1) \times n}$$

Here N_{batch} is the number of sampled states in one training batch. It is not a geometric dimension of the lattice. The second dimension is the channel dimension, which is one because the cosine-token matrix is treated as a single-channel structured input.

The second branch is the vector-feature branch. It takes

$$[\mathbf{g}_t, \mathbf{u}_t] \in \mathbb{R}^{n+5}$$

and processes it through an MLP.

The two branches are independent before fusion. The model does not first apply CNN and then apply MLP to the CNN output. Instead, the CNN branch and the MLP branch process different parts of the state in parallel. Their outputs are concatenated only after both branches have extracted their own features.

The column-direction convolution is

$$\mathbf{F}_{\text{col}} = \mathbf{K}_{k \times 1} * \mathbf{X}_C.$$

It scans down the token matrix and captures how correlations change along neighbouring basis indices.

The row-direction convolution is

$$\mathbf{F}_{\text{row}} = \mathbf{K}_{1 \times k} * \mathbf{X}_C.$$

It scans across the token matrix and captures how one group of basis vectors is correlated with nearby vectors.

The two feature maps are concatenated along the channel dimension:

$$\mathbf{F}_{\text{cat}} = \text{Concat}(\mathbf{F}_{\text{col}}, \mathbf{F}_{\text{row}}).$$

Then a fusion convolution is applied:

$$\mathbf{F}_{\text{fuse}} = \text{Conv}_{3 \times 3}(\text{Conv}_{1 \times 1}(\mathbf{F}_{\text{cat}})).$$

The 1×1 convolution mixes the directional channels, while the 3×3 convolution combines local information from both directions.

After convolution, adaptive pooling converts the feature map into a fixed-size vector

$$\mathbf{h}_C.$$

The vector-feature branch produces

$$\mathbf{h}_S = \text{MLP}([\mathbf{g}_t, \mathbf{u}_t]).$$

The final fused representation is

$$\mathbf{h}_t = \text{Fusion}([\mathbf{h}_C, \mathbf{h}_S]).$$

Axial convolution is used here as an architectural bias rather than as a theoretically optimal network. Compared with flattening the cosine-token matrix directly, it preserves row-wise and column-wise correlation patterns. Compared with a full two-dimensional convolution, it separates the two basis-index directions and reduces unnecessary parameter coupling. This makes it a reasonable feature extractor for the structured cosine-token input used in this thesis.

Algorithm 3 Forward Pass of AxialCNN-DDQN**Require:** State batch $\mathbf{S}$

- 1: Split $\mathbf{S}$ into $\text{vec}(\mathbf{T}(\mathbf{C}))$ and $[\mathbf{g}, \mathbf{u}]$
- 2: Reshape $\text{vec}(\mathbf{T}(\mathbf{C}))$ into $\mathbf{X}_C$
- 3: Compute column features:

$$\mathbf{F}_{\text{col}} = \text{Conv}_{k \times 1}(\mathbf{X}_C)$$

- 4: Compute row features:

$$\mathbf{F}_{\text{row}} = \text{Conv}_{1 \times k}(\mathbf{X}_C)$$

- 5: Concatenate:

$$\mathbf{F}_{\text{cat}} = \text{Concat}(\mathbf{F}_{\text{col}}, \mathbf{F}_{\text{row}})$$

- 6: Apply fusion convolution and adaptive pooling to obtain $\mathbf{h}_C$
- 7: Process $[\mathbf{g}, \mathbf{u}]$ through MLP to obtain $\mathbf{h}_S$
- 8: Concatenate $\mathbf{h}_C$ and $\mathbf{h}_S$, then apply fusion MLP to obtain $\mathbf{h}_t$
- 9: Compute value $V(\mathbf{s})$ and advantages $A(\mathbf{s}, \mathbf{a})$
- 10: Return

$$Q(\mathbf{s}, \mathbf{a}) = V(\mathbf{s}) + A(\mathbf{s}, \mathbf{a}) - \frac{1}{|\mathcal{A}|} \sum_{\mathbf{a}'} A(\mathbf{s}, \mathbf{a}')$$

NoisyNet Exploration

The agent uses NoisyNet exploration in the action-value estimation stage. NoisyNet is not applied to the cosine-matrix convolution branch or to the Gram-Schmidt feature extraction branch. Instead, it is embedded in the fully connected layers of the dueling DQN head, especially in the value stream and the advantage stream.

After the CNN branch and the vector-feature branch are fused, the network obtains the hidden representation

$$\mathbf{h}_t.$$

This representation is then passed into two streams:

$$V(\mathbf{s}_t) = V_\theta(\mathbf{h}_t),$$

and

$$A(\mathbf{s}_t, \mathbf{a}) = A_\theta(\mathbf{h}_t, \mathbf{a}).$$

In the implementation, the linear layers inside these two streams are replaced by noisy linear layers. A noisy linear layer can be written as

$$\mathbf{y} = (\mathbf{W}_\mu + \mathbf{W}_\sigma \odot \boldsymbol{\epsilon}_W) \mathbf{x} + (\mathbf{b}_\mu + \mathbf{b}_\sigma \odot \boldsymbol{\epsilon}_b),$$

where $\mathbf{W}_\mu$ and $\mathbf{b}_\mu$ are the learnable mean parameters, $\mathbf{W}_\sigma$ and $\mathbf{b}_\sigma$ are the learnable noise-scale parameters and $\boldsymbol{\epsilon}_W$, $\boldsymbol{\epsilon}_b$ are sampled noise variables.

Therefore, the Q-value output becomes stochastic during training:

$$Q(\mathbf{s}, \mathbf{a}; \theta, \epsilon) = V(\mathbf{s}; \theta, \epsilon) + A(\mathbf{s}, \mathbf{a}; \theta, \epsilon) - \frac{1}{|\mathcal{A}|} \sum_{\mathbf{a}'} A(\mathbf{s}, \mathbf{a}'; \theta, \epsilon).$$

This allows the agent to explore different BKZ actions through parameter noise rather than only through random action selection. In this thesis, this is useful because the action space contains many possible local BKZ blocks and purely random exploration may waste many steps on ineffective or expensive reductions.

During training, the noise variables are resampled periodically before action selection or network updates. During evaluation, the noise can be disabled or replaced by the mean parameters, so the policy becomes deterministic. Thus, NoisyNet is used as an exploration mechanism inside the DQN head, after feature fusion and before final Q-value computation.

Double DQN Output

The dueling architecture decomposes the Q-value into a state-value term and an advantage term:

$$Q(\mathbf{s}, \mathbf{a}) = V(\mathbf{s}) + A(\mathbf{s}, \mathbf{a}) - \frac{1}{|\mathcal{A}|} \sum_{\mathbf{a}'} A(\mathbf{s}, \mathbf{a}').$$

The value stream estimates the general quality of the current basis. The advantage stream estimates how much better a specific local BKZ action is compared with the average action.

The Double DQN target is

$$\begin{aligned} \mathbf{a}^* &= \arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}_{t+1}, \mathbf{a}'), \\ y_t &= r_t + \gamma Q_{\theta^-}(\mathbf{s}_{t+1}, \mathbf{a}^*), \end{aligned}$$

where θ is the online network and θ^- is the target network. This reduces the overestimation bias of standard DQN.

3.5 Chapter Summary

This chapter presented the methodology of the proposed DRL-controlled local BKZ framework. The raw integer basis $\mathbf{B}$ is reparameterized into a stable geometric state consisting of the cosine-token matrix $\mathbf{T}(\mathbf{C})$, the normalized Gram-Schmidt length profile and five scalar indicators. This representation is designed to improve numerical stability and expose geometric information relevant to local BKZ scheduling, rather than to reduce the number of input parameters.

The chapter then formulated local BKZ scheduling as a finite-horizon decision problem. The agent is trained on multiple cached LLL-reduced seeds and learns to select local actions $\mathbf{a} = (\beta, pos)$. The reward prioritizes first-vector improvement toward $\rho(\mathbf{B}) \leq 1.05$, while orthogonality improvement is used as an auxiliary signal. The AxialCNN-DDQN architecture processes the cosine-token matrix and vector features through parallel branches and NoisyNet exploration is embedded in the dueling DQN head for action selection.

Chapter 4

Implementation

4.1 Chapter Overview

This chapter describes the code-level implementation and experimental setup of the whole deep reinforcement learning framework. This chapter focuses on the implementation pipeline, data transfer between modules, cached seed initialization, environment execution, training configuration, baselines, training log and evaluation protocol.

4.2 Implementation Pipeline and Data exchange

The implementation is divided into three layers: the C++/fplll backend, the Rust interface and the Python reinforcement learning system. The C++ layer performs high-precision lattice reduction, including LLL and BKZ 2.0. The Rust layer manages the matrix pool, calls the backend, extracts geometric features and exposes Python bindings. The Python layer implements all the agent functions:

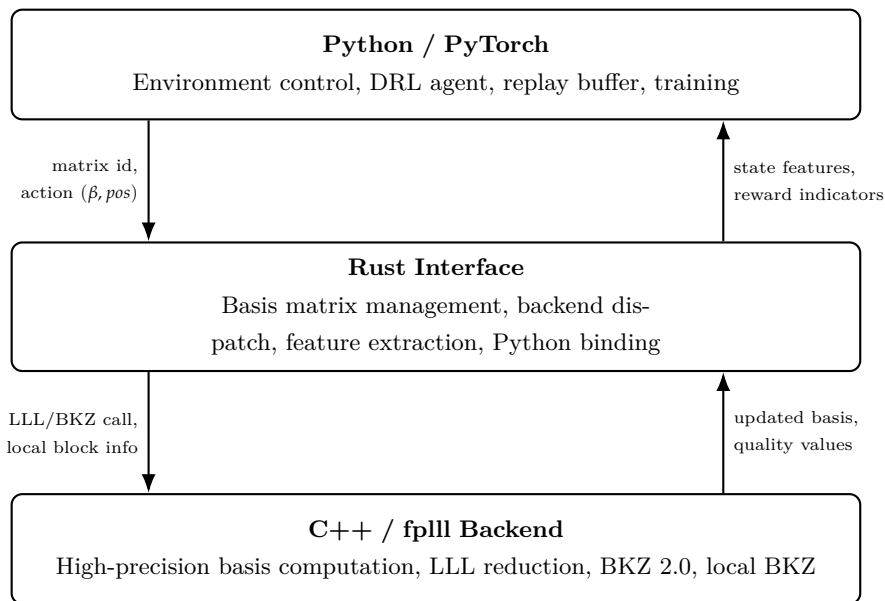


Figure 4.1: Code-level implementation pipeline.

The key design principle is that the raw integer basis is kept inside the backend and is not repeatedly transferred to Python. Python only receives compact floating-point geometric features. This reduces data-transfer overhead and avoids exposing extremely large integer entries to the neural network.

The data flow of the implementation is summarized in Table 4.1. The most important point is that different layers exchange different types of data. The backend stores and modifies the high-precision basis, while the Python agent only sees the reduced state vector and scalar indicators.

Table 4.1: Data transferred between implementation modules.

Direction	Data type	Purpose
Python $\rightarrow$ Rust	raw basis when initial loading	The SVP Challenge basis is parsed in Python and sent to the backend during initialization.
Python $\rightarrow$ Rust	matrix-pool id	The environment refers to the current basis instead of copying the full matrix.
Python $\rightarrow$ Rust	action index or (β, p)	The selected DRL action is decoded into a local BKZ block size and starting position.
Rust $\rightarrow$ C++	reduction command	Rust calls the C++ backend to apply LLL, local BKZ 2.0.
C++ $\rightarrow$ Rust	updated basis	The high-precision bases is modified in the backend and remain stored in the matrix pool.
Rust $\rightarrow$ Python	cosine matrix and GS features	Rust extracts the geometric features.
Rust $\rightarrow$ Python	norm/GH ratio, defect, max cosine	These scalar indicators are used for reward computation, logging and stopping conditions.
Python $\rightarrow$ PyTorch	state vector	The environment sends the constructed state vector to the AxialCNN-DDQN agent.
PyTorch $\rightarrow$ Python	action index	The agent returns the selected action index for the next local BKZ operation.

This design separates exact lattice manipulation from neural-network computation. The C++ backend handles integer lattice reduction, Rust handles feature extraction and memory management and Python handles learning and decision-making.

4.3 Cached LLL Initialization

For each tested dimension, several SVP Challenge seed bases are loaded before training. In the current implementation, the seed set is

$$\mathcal{I} = \{0, 1, 2, 3, 5, 7, 9, 11, 13\}.$$

The current experiments focus on dimensions 50 and 51. Training is performed separately for each dimension and all selected seeds of that dimension are used together during training.

Each raw basis is first reduced by LLL and then stored as a cached initialization. This means that the expensive initial LLL preprocessing does not need to be repeated at every episode reset. The cached basis is treated as the standard starting point of an episode.

At reset time, the environment randomly selects one seed from the cached seed set and clones its LLL-preprocessed basis. The cloned basis becomes the mutable current basis of the episode. The cached original copy remains unchanged and can be reused by other episodes or workers.

4.4 Environment Execution and State Construction

The environment is responsible for connecting the learned policy with the backend reduction procedure. Each environment instance maintains the current matrix-pool identifier, the current seed index, the episode step number, cached quality indicators, recent action history and episode-best information. The raw integer basis is not stored or modified directly in Python. Instead, Python only keeps the identifier of the current basis and communicates with the Rust interface when a reduction or feature extraction operation is required.

Each action corresponds to one local BKZ operation. In the implementation, an action is represented by a pair (β, p) , where β is the BKZ block size and p is the starting position of the local block. The action space is generated once when the environment is initialized. For dimension n , the maximum block size is set to $\min(\lfloor 0.8n \rfloor, 50)$ and the minimum block size is set to $\max(8, \lfloor 0.15n \rfloor)$. The candidate block sizes are generated by geometric spacing between these two values and duplicate values after rounding are removed. For each block size, candidate starting positions are generated with step size approximately equal to half of the block size. The first valid position is always included. This gives the agent a finite list of local BKZ actions and the DQN output dimension is equal to the number of actions in this list.

At each step, the agent first outputs an action index. The Python environment decodes this index into (β, p) , sends the current matrix-pool identifier and the action parameters to Rust and Rust calls the C++/fplll backend to perform the local BKZ 2.0 operation. After the backend operation, the environment may apply LLL stabilization. In the implemented setting, LLL is applied periodically after every three local BKZ steps. This stabilization is part of the environment transition and is not selected by the agent.

After the backend finishes the reduction operation, Rust extracts the updated geometric features and sends them back to Python. The state vector is then assembled from three parts: the flattened cosine-token matrix, the normalized Gram-Schmidt profile and five scalar indicators. The cosine-token part is placed at the beginning of the state vector. Inside the neural network, this part is reshaped into a two-dimensional matrix and processed by the axial convolutional branch.

The Gram-Schmidt profile and scalar indicators are processed by a separate fully connected branch. The scalar indicators include the maximum cosine value, the normalized logarithmic defect, the normalized logarithmic ratio, a slope-like summary of the Gram-Schmidt profile and the normalized step progress. The step-progress feature tells the agent how much of the episode budget has already been used.

The reward is also computed in the Python environment after the new backend features are returned. The implementation uses cached previous values and newly returned values. The main cached quantities are the logarithmic first-vector norm, the maximum cosine value and the logarithmic defect. After a backend step, the environment computes the improvement in first-vector length, the improvement in maximum cosine value and the improvement in defect. It also subtracts a cost term depending on the selected block size. Larger block sizes are more expensive and therefore receive a larger cost penalty. In addition, the implementation includes several practical shaping terms, including a global-best bonus, an episode-best bonus, a small early-position bonus, a terminal penalty for unsuccessful episodes and a repeated-action penalty. The final reward is clipped to a fixed interval to avoid unstable Q-value updates caused by rare large bonuses or penalties.

At the end of an episode, the implementation applies a terminal large-block BKZ polish at the beginning of the basis. The final block size is set to $\min(n, 40)$. This step forces a stronger final reduction of the leading block. During an episode, the agent may improve local geometry in different parts of the basis, but these improvements do not always immediately shorten the first vector. The final large-block BKZ call focuses on the first part of the basis and converts accumulated basis-profile improvements into a stronger final short-vector result[19]. The final episode ratio is recorded after this terminal polish.

4.5 Training Configuration

Training uses multiple parallel environment workers. Each worker maintains its own current matrix-pool identifier and episode state. The main process sends actions to the workers and receives next states, rewards, terminal indicators and diagnostic information. In the current implementation, the number of parallel workers is 12. Each worker samples from the same cached seed set but clones its own mutable basis. Therefore, different workers can explore different reduction paths independently.

The maximum number of local BKZ decisions in one episode is $3n$. For example, dimension 50 uses at most 150 agent-controlled local BKZ decisions per episode. This gives each episode a fixed reduction budget and makes the comparison between different policies more controlled.

The replay buffer uses prioritized experience replay with capacity 50000. The batch size is 128. States are stored in half precision inside the replay buffer to reduce memory usage and are converted back to floating-point tensors during training. The optimizer is AdamW with learning rate 6×10^{-5} and weight decay 10^{-4} . A cosine annealing learning-rate scheduler is used, with minimum learning rate 10^{-6} . The discount factor is 0.99 and the soft target-network update coefficient is 0.0025. Gradient clipping is applied with maximum norm 0.75.

The agent uses NoisyLinear exploration together with a small epsilon-greedy component. During training, the NoisyLinear noise is reset before action selection. The epsilon value decreases during

training but remains at least 0.05. During final evaluation, the policy selects actions greedily from the learned Q-values.

The main success threshold is $\rho(B) \leq 1.05$, where $\rho(B)$ is the first-vector norm divided by the Gaussian Heuristic. For each tested dimension, training uses nine seeds. Training stops when at least six of the nine seeds reach the threshold. Training can also stop when the maximum episode budget is reached or when no new global improvement is observed for a fixed patience window. This prevents unnecessary computation after the policy has stopped improving.

4.6 Baselines, Evaluation and Logging

The proposed DRL-controlled local BKZ policy is compared with three baseline methods. The first baseline is LLL-only reduction. This baseline applies LLL preprocessing and evaluates the resulting basis without any BKZ scheduling. It measures the quality obtained from the simplest reduction pipeline. The second baseline is a fixed BKZ 2.0 schedule. This baseline applies predetermined BKZ operations without using a learned policy and it is used to compare adaptive scheduling with manually specified reduction behavior. The third baseline is random local BKZ. It samples actions uniformly from the same action list used by the DRL agent. It uses the same local BKZ backend, periodic LLL stabilization and terminal large-block BKZ polish. This baseline separates the effect of learned scheduling from the effect of simply using local BKZ actions. The proposed method is the DRL-controlled local BKZ policy. During evaluation, the trained agent selects the action with the largest predicted Q-value and no replay update or exploration noise is used in the final reported evaluation.

The main evaluation metric is the final norm-to-Gaussian-Heuristic ratio. Lower values indicate better short-vector quality and the threshold 1.05 is used as the main success target. The implementation also records the logarithmic orthogonality defect, maximum cosine value, minimum cosine value, final first vector, selected actions, episode reward, training loss and runtime-related information. These values are saved for later analysis.

Action distribution is recorded to show how the agent uses different block sizes and starting positions. This helps interpret whether the policy mainly focuses on the leading part of the basis or also improves middle blocks before the final polish. Cosine-matrix heatmaps are generated in Chapter 5 to compare the geometric structure before and after reduction. The heatmaps are not used as part of training. They are used as an interpretability tool for evaluating whether the learned policy reduces local correlation patterns in the basis.

All generated figures can be stored in the same figure directory used by the rest of the thesis. For example, the directory structure can be written as `figures/`, with files such as `code_pipeline.png`, `training_curve.png`, `heatmap_111.png` and `heatmap_drl.png`. If the implementation pipeline is drawn using TikZ, no external image file is required. If the diagram is exported as a PNG file, it can be inserted using `\includegraphics[width=0.9\textwidth]{Gragh/code_pipeline.png}`. In this thesis, the TikZ version is preferable for the code pipeline because it produces a cleaner and more consistent academic figure.

4.7 Experimental Protocol

For each tested dimension, the experiment follows the same procedure. First, the SVP Challenge bases are loaded. Second, each raw basis is LLL-preprocessed and stored in the matrix pool. Third, the action space is constructed from the dimension-dependent block-size and position candidates. Fourth, the DRL agent is trained using parallel environments. Fifth, terminal large-block BKZ polish is applied at the end of each episode. Finally, the learned policy is compared with the baseline methods using the same evaluation metrics.

The complete implementation protocol is therefore: load seed bases, apply cached LLL preprocessing, clone one cached basis at reset time, construct the geometric state, select a local BKZ action using the agent, call the backend through Rust, extract new features, compute reward, store transition, update the Q-network, apply terminal polish, record results and compare with baselines.

4.8 Chapter Summary

This chapter described the code implementation and experimental setup of the proposed framework. The implementation uses C++/fplll for LLL and BKZ 2.0, Rust for matrix-pool management and feature extraction and Python/PyTorch for the DRL environment and agent. The raw integer basis remains inside the backend, while Python receives only compact geometric features and scalar indicators. The numerical results produced by this pipeline are analyzed in Chapter 5.

Chapter 5

Experimental Results and Brief Analysis

This chapter presents the experimental results obtained from the implementation described in Chapter 3 & 4. The purpose of this chapter is to evaluate whether the proposed DRL-controlled local BKZ framework can efficiently improve the quality of LLL-preprocessed SVP Challenge bases. The main metric is the norm-to-Gaussian-Heuristic ratio $\rho(B) = \|b_1\|/GH(\mathcal{L})$, where a smaller value indicates a shorter best vector. The target threshold used in this chapter is $\rho(B) \leq 1.05$.

The experiments are based on the result files saved during training. Each file records the initial LLL-preprocessed state and the new best states found during training. The available logs support the analysis of training dynamics, full DRL-BKZ performance, initial-versus-final basis quality, seed-level variability, logarithmic orthogonality defect, maximum cosine value and representative cosine-heatmap comparison. Some additional experiments, such as action-distribution analysis and runtime profiling still require additional logs or separate runs. As a result, they are therefore not included in the current chapter.

5.1 Training Dynamics

Figure 5.1 shows the best learning curve for each tested dimension. The curves corresponds to the seed that achieved the smallest final ratio among the parsed result files. The curve is shown as a step plot because the result files only record new best events. Therefore, a horizontal segment means that no better result was recorded during that interval, while a downward jump means that the training process discovered a better reduced basis.

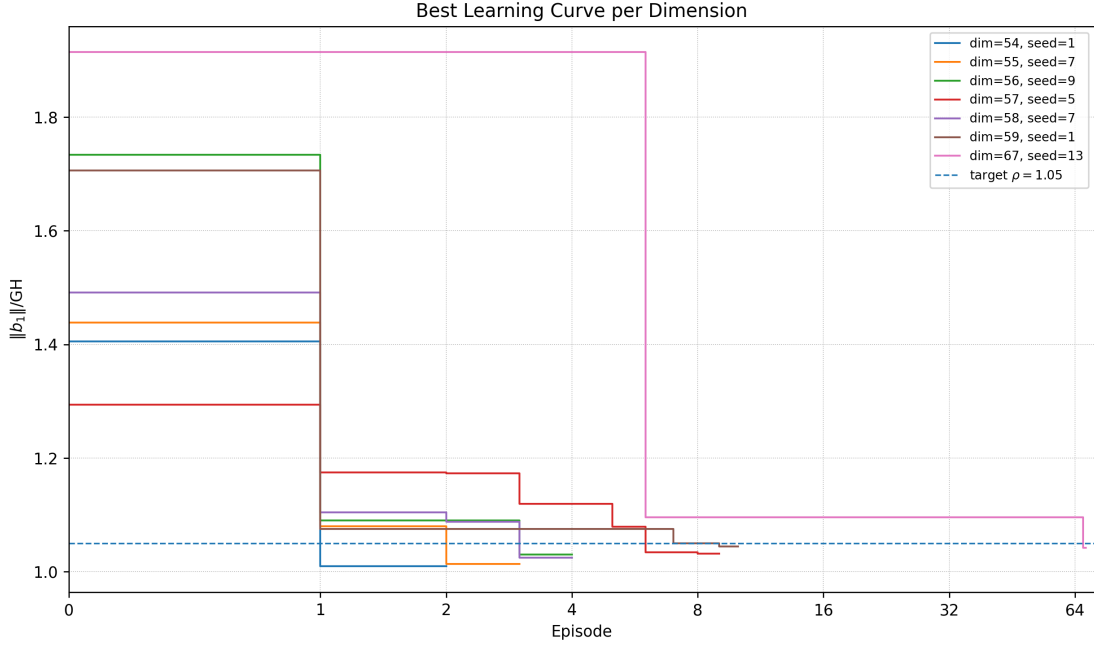


Figure 5.1: Best seed's learning curve per dimension.

The learning curves show that most dimensions obtain large improvements within the first few episodes. For dimensions 55, 56, 57 and 58, the best curves fall close to the target threshold after only a small number of new-best updates. Dimension 67 behaves differently: its best curve remains high for a longer period and improves much later. This indicates that higher-dimensional or harder instances may require more exploration before the learned local BKZ schedule finds a strong reduction strategy.

Figure 5.2 reports the experimental success count during training. This plot should be interpreted as an experimental diagnostic rather than a complete training trace, because it is reconstructed from the saved new-best result files. A seed is counted as successful once its best recorded ratio reaches $\rho \leq 1.05$.

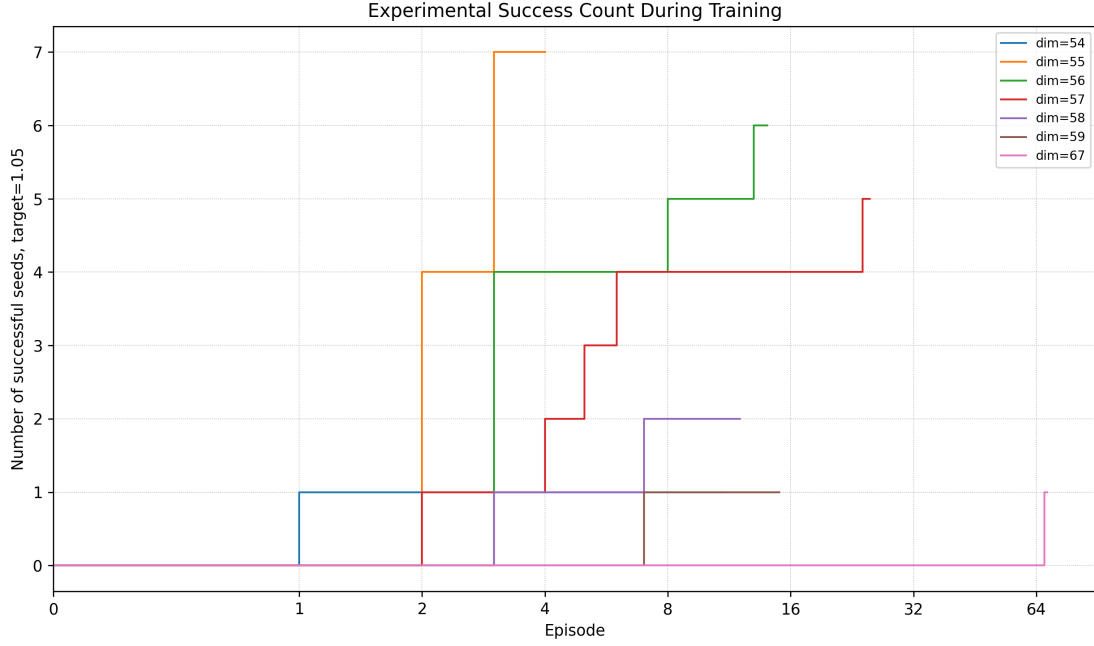


Figure 5.2: Experimental success count during training.

The success-count curve confirms that the learned policy is not only improving a single isolated seed. Dimension 55 reaches 7 successful seeds, dimension 56 reaches 6 successful seeds, dimension 57 reaches 5 successful seeds and dimension 58 reaches 2 successful seeds under limited computing resources. This difference suggests that the difficulty is not determined only by dimension: the seed distribution, initial LLL quality and the local geometry of each basis also strongly affect the final result.

5.2 Initial LLL and Final DRL-BKZ Comparison

Figure 5.3 compares the mean ratio after initial LLL preprocessing with the mean ratio after the final DRL-BKZ result. The dimension 59 are omitted from this figure for visual clarity because their final results contain stronger failed-seed effects. The full numerical results including these dimensions are still reported in Table 5.1.

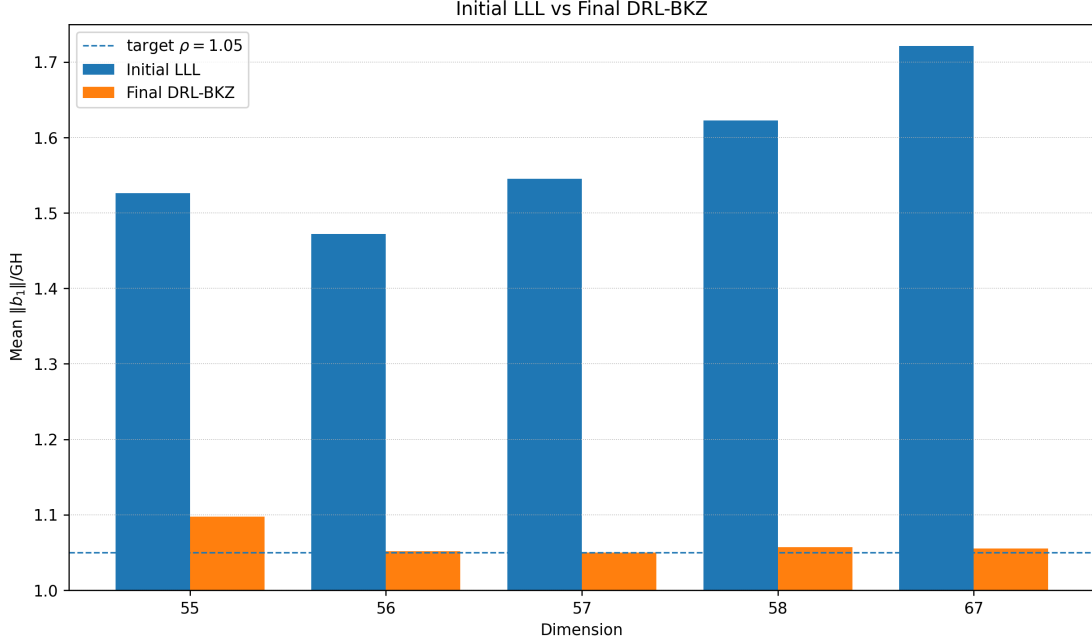


Figure 5.3: Mean Ratio ($\mathbb{E}[\rho(\mathbf{B})]$) Comparison between initial LLL and final DRL-BKZ.

The comparison shows a clear reduction from the LLL-preprocessed state to the final DRL-BKZ state. The improvement is especially important because LLL is used as the starting point of all episodes. Therefore, the final result is not merely the effect of standard preprocessing. It reflects additional basis improvement produced by the learned sequence of local BKZ operations and the terminal large-block BKZ polish. However, the remaining gap above 1.05 in several dimensions also shows that the current training protocol is not uniformly successful for all seeds.

5.3 Full DRL-BKZ Results

Table 5.1 reports the full results for all parsed dimensions. The table includes the number of seeds, success count, mean ratio after initial LLL, mean final ratio after DRL-BKZ, standard deviation of the final ratio, best ratio, worst ratio and the corresponding orthogonality indicators.

Table 5.1: Main results of the DRL-BKZ framework under limited computing resource.

Dim	Seeds	Success	Initial Mean ρ	Final Mean ρ	Final Std	Best ρ	Worst ρ	Initial D	Final D	Initial $C_{\max}$	Final $C_{\max}$
55	10	7	1.526076	1.097721	0.157492	1.013784	1.565725	61.757678	51.712363	0.633711	0.556808
56	9	6	1.471896	1.051791	0.018997	1.030674	1.085817	63.424211	51.875380	0.616723	0.531639
57	9	5	1.545296	1.050013	0.014825	1.031721	1.073915	66.161940	54.162343	0.647389	0.570016
58	9	2	1.622807	1.056855	0.016743	1.024788	1.081473	68.665870	55.085114	0.607023	0.566574
59	9	1	1.567489	1.195177	0.222544	1.044528	1.704472	71.347489	62.169043	0.637183	0.632975
67	2	1	1.721165	1.055030	0.013104	1.041926	1.068133	91.586409	72.699478	0.669576	0.600208

The table shows that the proposed method consistently reduces the mean ratio compared with the initial LLL state. The strongest mean-ratio improvement appears in dimension 67, where the mean ratio decreases by approximately 0.666136, although this dimension currently contains only two parsed seeds. Among the dimensions with nine or more seeds, dimension 58 has the largest mean-ratio improvement, decreasing by approximately 0.565952. Dimensions 56 and

57 are the most stable near the target threshold, with final standard deviations 0.018997 and 0.014825, respectively.

The success count gives a stricter view of robustness. Dimension 55 reaches 7/10 successful seeds, dimension 56 reaches 6/9, dimension 57 reaches 5/9 and dimension 58 reaches 2/9. The best ratios are all close to or below the target threshold, with the best results being 1.009466 for dimension 54, 1.013784 for dimension 55, 1.030674 for dimension 56, 1.031721 for dimension 57, 1.024788 for dimension 58, 1.044528 for dimension 59 and 1.041926 for dimension 67. This indicates that the framework can find strong reductions in every tested dimension, although it does not yet solve all seeds uniformly.

Figure 5.4 visualizes the final mean ratio and standard deviation for the main displayed dimensions. The error bars show the variation across seeds.

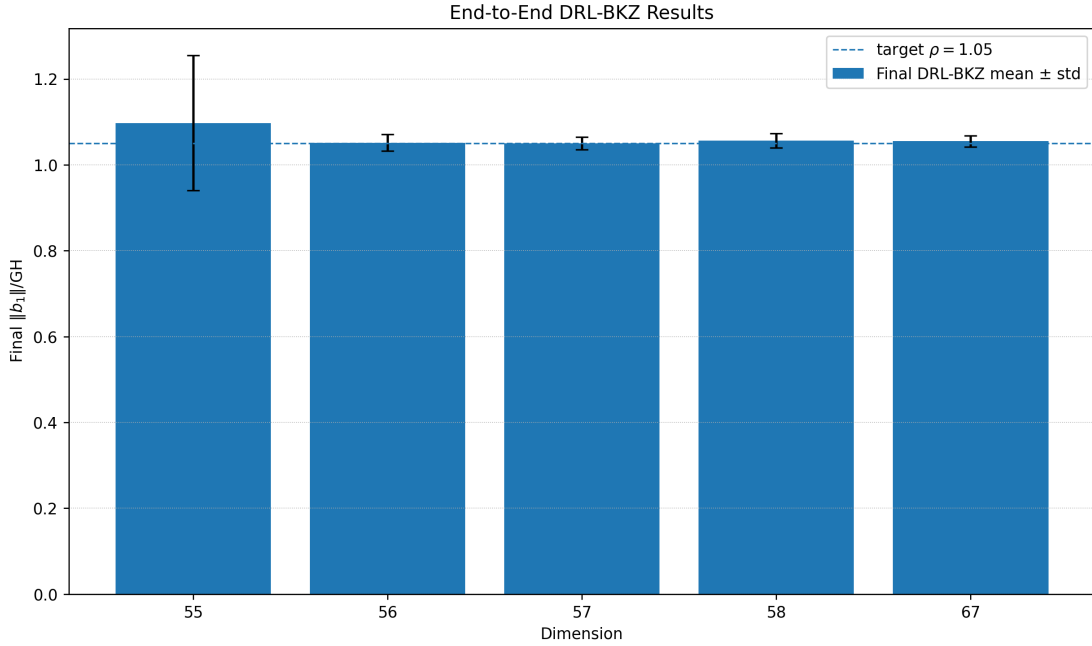


Figure 5.4: DRL-BKZ final ratios with standard deviation.

The figure shows that dimensions 56, 57, 58 and 67 are all very close to the target threshold on average. Dimension 55 has a higher mean and larger standard deviation because one seed remains much worse than the others. This seed-level variability is important because it shows that reporting only the best result would overestimate the robustness of the method.

Figure 5.5 further shows the per-seed final ratios. Each point represents one seed. The dashed line again denotes the target threshold.

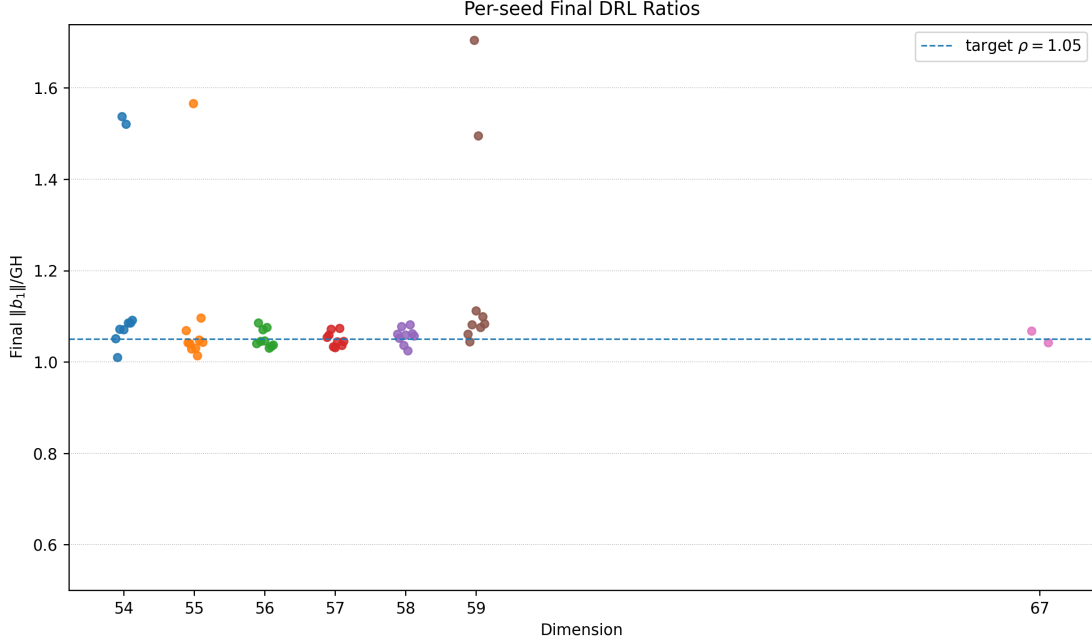


Figure 5.5: Per-seed final DRL-BKZ ratios. Each point corresponds to one seed.

The scatter plot shows that most successful or near-successful results are concentrated close to 1.05, while several outlier seeds remain significantly higher. These outliers are visible in dimensions 54, 55 and 59. This supports the conclusion that the current framework is effective for many seeds but still sensitive to seed-specific lattice geometry and training stability.

5.4 Orthogonality Metrics

Besides the first-vector ratio, the implementation also records geometric indicators of basis quality. This section analyzes the logarithmic orthogonality defect $D(B)$ and the maximum cosine value $C_{\max}(B)$. These metrics are important because the reward function does not only encourage shortening of b_1 , but also includes terms related to global orthogonality and local pairwise correlation.

Figure 5.6 compares the mean logarithmic orthogonality defect before and after DRL-BKZ reduction.

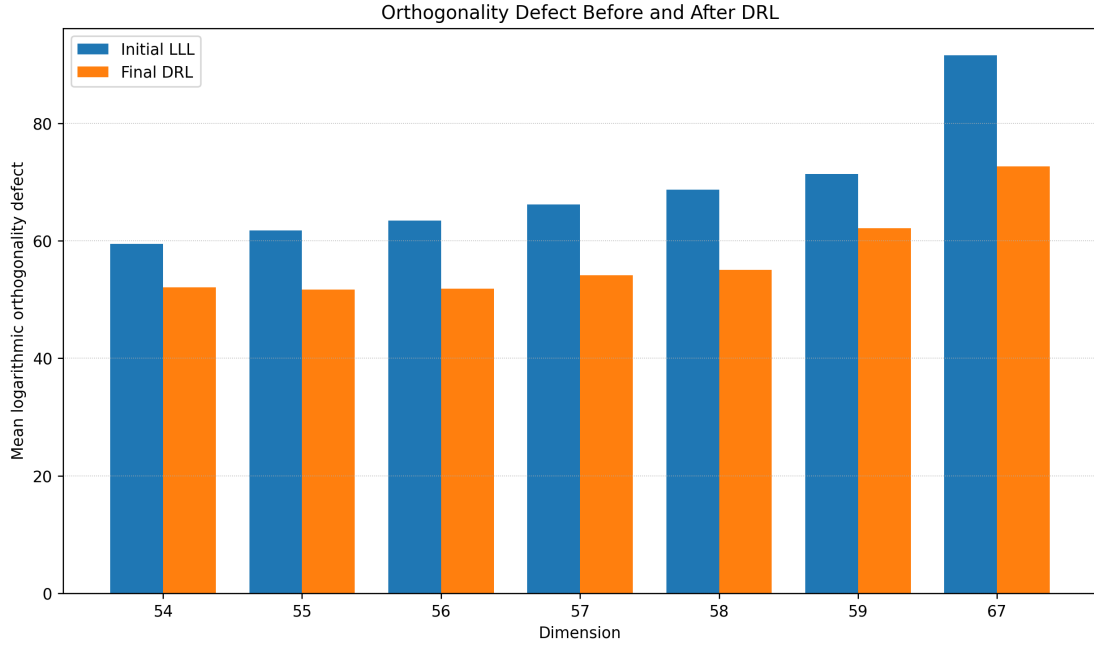


Figure 5.6: Mean logarithmic orthogonality defect ($\mathbb{E}[\delta(\mathbf{B})]$) before and after DRL-BKZ reduction.

The logarithmic orthogonality defect decreases in all tested dimensions.

For dimension 55, it decreases from 61.757678 to 51.712363.

For dimension 56, it decreases from 63.424211 to 51.875380.

For dimension 57, it decreases from 66.161940 to 54.162343.

For dimension 58, it decreases from 68.665870 to 55.085114. The decrease indicates that the learned local BKZ schedule improves not only the first vector but also the overall orthogonality structure of the basis.

Figure 5.7 compares the mean maximum cosine value before and after reduction.

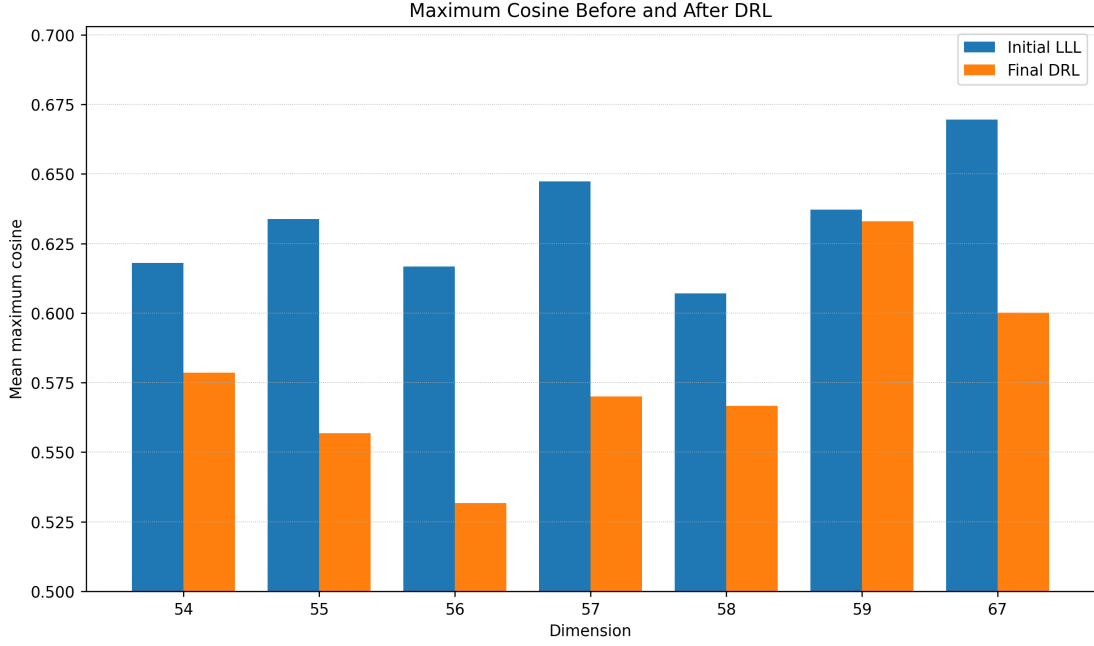


Figure 5.7: Mean maximum cosine before and after DRL-BKZ reduction.

The maximum cosine value also decreases in all dimensions.

For dimension 55, the mean maximum cosine decreases from 0.633711 to 0.556808.

For dimension 56, it decreases from 0.616723 to 0.531639.

For dimension 57, it decreases from 0.647389 to 0.570016.

For dimension 58, it decreases from 0.607023 to 0.566574.

Dimension 59 only shows a very small mean improvement, from 0.637183 to 0.632975, which is consistent with its weaker final ratio performance.

Together, the orthogonality defect and maximum cosine results support the role of the cosine-based state representation and orthogonality-related reward terms. The agent is not only shortening b_1 through final polishing, but is also improving the geometric quality of the basis in many dimensions. However, the improvement of maximum cosine is not uniform, which suggests that the current policy still prioritizes short-vector improvement more strongly than complete removal of local angular correlation.

5.5 Cosine Heatmap Comparison

To complement the scalar orthogonality indicators, Figure 5.8 provides a visual comparison of the cosine structure of representative successful seeds. Each row corresponds to one seed and the three columns show the initial LLL-preprocessed basis, the fixed Local-BKZ baseline and the final DRL-BKZ result, respectively. The heatmaps visualize the lower-triangular part of the shifted cosine matrix used in the implementation. Brighter regions indicate stronger pairwise angular correlation, while darker regions indicate better local orthogonality.

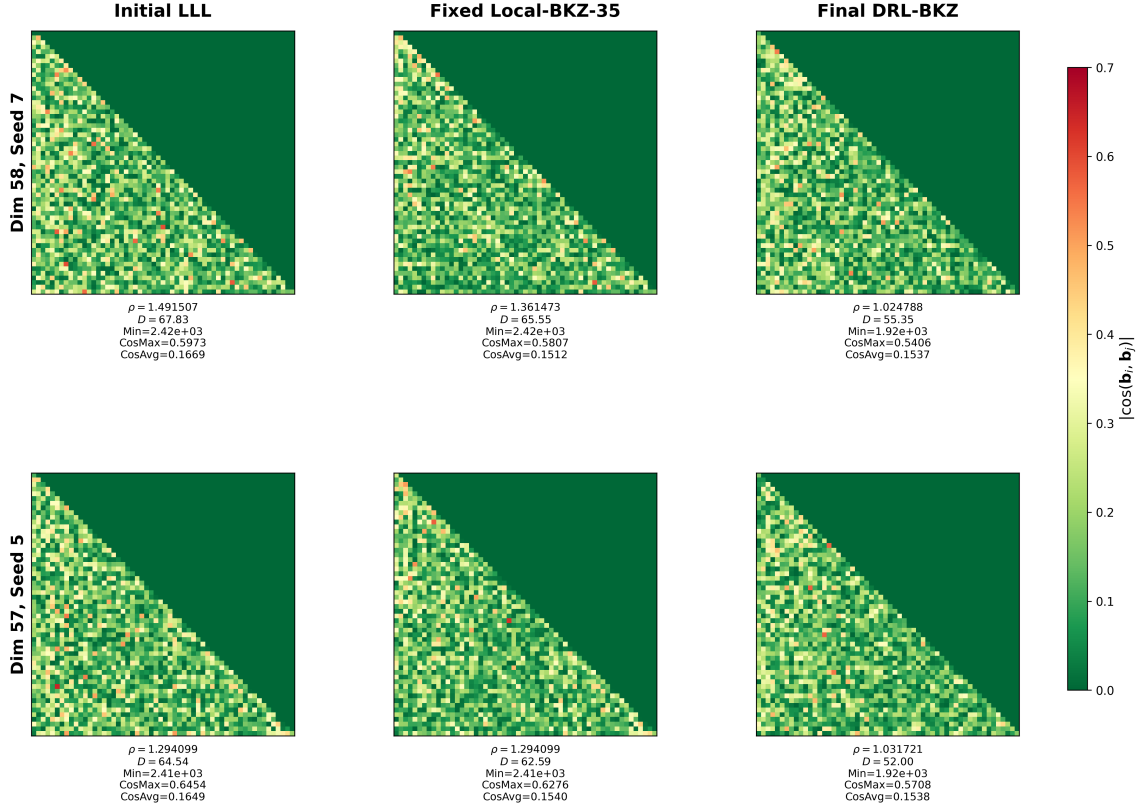


Figure 5.8: Representative cosine-heatmap comparison among Initial LLL, Fixed Local-BKZ-40 and Final DRL-BKZ.

Several observations can be made from Figure 5.8. First, the overall results are good: the final DRL-BKZ basis in both examples achieves a substantially smaller $\rho(B)$ than the initial LLL basis and the corresponding logarithmic orthogonality defect also decreases noticeably. For instance, in dimension 58, seed 7, the ratio decreases from 1.491507 after initial LLL to 1.242907 after the fixed Local-BKZ baseline and further to 1.024788 after DRL-BKZ, while the orthogonality defect decreases from 67.83 to 64.39 and then to 55.35. Similarly, in dimension 57, seed 5, the ratio decreases from 1.294099 after initial LLL to 1.031721 after DRL-BKZ, while the defect decreases from 64.54 to 52.00.

Second, only a very small number of vectors remain visibly poor in orthogonality. In the initial LLL and fixed Local-BKZ panels, fixed strategy tends to reduce the non-orthogonality to the minimum number of vector pairs as much as possible. In contrast, in the final DRL-BKZ panels these stronger correlations are generally weakened and become more localized. Qualitatively, the learned reduction tends to disperse a very poor basis configuration into several better vectors and suppress the global correlation level, while the remaining difficult interactions become concentrated in a relatively small number of columns.

This observation is consistent with the scalar metrics in the previous subsection. The logarithmic orthogonality defect decreases substantially in the final DRL-BKZ basis and the maximum cosine is also reduced relative to the initial LLL basis, although the exact improvement may vary from seed to seed. Therefore, the heatmap results provide visual evidence that the learned local BKZ schedule improves basis geometry in addition to shortening the first vector.

At the same time, the heatmaps also suggest a limitation of the current policy. Although the final DRL-BKZ basis is much better overall, the remaining difficult structures are still concentrated in several localized columns. This indicates that the current local scheduling policy can identify and improve many problematic regions, but may still leave a small number of concentrated hard interactions unresolved. This observation motivates a natural direction for Chapter 6: future work may introduce more dispersed row or block selection strategies, so that the reduction effort can be distributed across several separated problematic regions instead of repeatedly focusing on only one local area.

5.6 Scope of Available Results

The current result files support the analysis presented above, but they do not contain all information needed for the full experimental plan. However, the saved summary files do preserve representative final basis matrices for successful seeds, which makes it possible to reconstruct representative cosine heatmaps for qualitative comparison. Therefore, action-distribution analysis, runtime profiling and detailed computational-cost analysis still cannot be reconstructed from the current files alone, while representative heatmap analysis is now available.

For this reason, the current chapter focuses on the results that can be directly supported by the available logs: training dynamics, initial-versus-final ratio improvement, DRL-BKZ performance, seed-level variability, orthogonality defect, maximum cosine and representative cosine-heatmap comparison. The remaining comparisons are discussed as limitations and future work in Chapter 6.

5.7 Chapter Summary

This chapter evaluated the proposed DRL controlled local BKZ framework on available SVP Challenge result files. The results show that the method consistently improves the mean Norm/GH ratio compared with the initial LLL-preprocessed state. In tested dimensions, the final ratios are mostly around the target threshold $\rho = 1.05$.

The training curves show that the best ratio generally decreases during training, while the experimental success count increases for several dimensions. The total results show that dimensions 56, 57, 58 and 67 finish close to the target threshold, while dimensions 55 and 59 contain more visible outlier seeds. The orthogonality defects further show that the logarithmic orthogonality defect decreases in all tested dimensions and that the maximum cosine value decreases in most dimensions.

Overall, the results suggest that the learned local BKZ schedule can improve both short-vector quality and basis geometry beyond fixed BKZ. At the same time, the remaining failed seeds and missing ablation baselines show that the current framework should be interpreted as an adaptive method which avoid overfitting.

Chapter 6

Discussion and Future Work

6.1 Discussion and Limitations

Overall Interpretation

The experimental results show that the proposed DRL-controlled BKZ can improve LLL-preprocessed SVP Challenge bases under the current implementation. The final DRL-BKZ results reduce the Norm/GH ratio in all tested dimensions and often move the first vector close to the target threshold $\rho(B) \leq 1.05$. The orthogonality defect and maximum cosine value also decrease in most cases, which indicates that the learned policy improves not only $\mathbf{b}_1$ but the geometric quality of the basis.

The heatmap comparison gives a more detailed explanation. Compared with the initial LLL basis and the fixed Local-BKZ baseline, the final DRL-BKZ basis usually has weaker and more localized high-correlation regions. This suggests that the learned schedule can reduce broad non-orthogonal structures. However, some hard interactions still remain concentrated in a few columns. Therefore, the method is effective as an adaptive scheduling framework. However, it does not completely remove all local correlation hotspots.

Experimental and Logging Limitations

The experiments are still limited in dimension range, number of seeds and computing budget. The current policy is also trained separately for each dimension, so it does not yet demonstrate cross-dimensional generalization.

Another limitation is incomplete experimental logging. The current logs are sufficient for final ratios, orthogonality metrics and representative heatmaps, but they do not contain complete step-level action traces or accurate more specific statistics. Therefore, the computational cost and the full action distribution cannot be completely analyzed in this version. Some ablation experiments, such as AxialCNN versus MLP and full-state versus no-cosine-state comparison, also require additional independent runs.

Residual Local Correlation

The heatmap results show that DRL-BKZ can improve the global basis structure, but the remaining bad correlations are not always eliminated. Instead, they may become concentrated in a few columns. This means that the current contiguous local-block action

$$\mathbf{a}_t = (\beta_t, pos_t)$$

may not be flexible enough to handle several separated difficult regions. This observation motivates future work on more dispersed block or row selection.

6.2 Future Work

Post-target Orthogonality Optimization

One important future direction is to continue optimization after the short-vector target has been reached. In the current implementation, the main target is

$$\rho(B) \leq 1.05.$$

Once this target is achieved, the agent can shift more attention from shortening $\mathbf{b}_1$ to improving orthogonality.

There are two feasible strategies. The first strategy is to adjust the reward weights after reaching the ratio threshold. Let

$$\Delta_\rho = \max(0, 1.05 - \rho(B))$$

Then the weights can be increased by

$$w'_\rho = w_\rho + c_\rho \Delta_\rho, \quad w'_D = w_D + c_D \Delta_\rho, \quad w'_C = w_C + c_C \Delta_\rho.$$

In this design, when the ratio becomes lower than the target, the margin Δ_ρ becomes larger, and the agent is encouraged to optimize $D(B)$ and $C_{\max}(B)$ more strongly.

The second strategy is to copy the trained network after it reaches the target ratio and fine-tune this copied network with larger orthogonality reward weights. In this way, the first stage learns how to reach a good short vector, while the second stage specializes in improving the remaining basis geometry. This two-stage design may be more stable than changing the reward weights inside a single training process.

Dispersed Block and Row Selection

The current local BKZ action always reduces one contiguous block. However, the heatmaps show that the remaining hard correlations may appear in several separated columns. Future work can therefore introduce dispersed row or block selection. Instead of only using

$$a_t = (\beta_t, pos_t),$$

the agent may select a set of problematic indices:

$$a_t = (\beta_t, S_t), \quad S_t = \{pos_{1t}, pos_{2t}, \dots, pos_{kt}\}.$$

This would allow the policy to focus on several separated high-correlation regions.

Time-series Reward Modelling

Another future direction is to study time-series information in the reward. A local BKZ operation may not immediately reduce $\rho(B)$, but it may improve the basis profile and reduce the defect after several later steps. Therefore, future experiments should test time-series models for both the ratio sequence and the defect sequence.

Possible models include MS, ES, EWMA, GARCH and so on. For example, a smoothed defect signal can be written as

$$\bar{D}_t = \alpha D_t + (1 - \alpha) \bar{D}_{t-1}.$$

The same idea can be applied to $\rho(B)$ or $C_{\max}(B)$. Different models should be compared experimentally to determine whether temporal reward information improves training stability and final reduction quality.

Multi-dimensional Training

The current agent is trained separately for each dimension. Future work can train a dimension-conditioned policy on several dimensions together. This may further improve generalization and reduce the training cost for new dimensions. A possible implementation is to add a dimension embedding so that the neural network can process variable-size lattice states.

Sieving-assisted BKZ

Finally, we can integrate sieving with BKZ. Inside BKZ, enumeration is limited. The local SVP subroutine can be made stronger than ordinary enumeration when sieving is integrated into BKZ, which may lead to more aggressive reduction and better basis quality. This may lead to stronger reduction quality than BKZ2.0 scheduling alone.

Chapter 7

Conclusion

This thesis studied the use of Deep Reinforcement Learning to optimize local BKZ scheduling for SVP-related lattice reduction. The main idea is to replace a fixed manually designed reduction schedule with an adaptive policy that observes the basis’s geometry feature and selects local BKZ actions.

This framework reparameterizes the raw integer basis into a stable geometric state. The state consists of a cosine-token matrix, a normalized Gram-Schmidt length profile and several scalar indicators including the Norm/GH ratio, logarithmic orthogonality defect and maximum cosine value. This avoids directly feeding large integer entries into the neural network and provides more useful information for local BKZ scheduling.

The agent is implemented as an AxialCNN-Dueling-DDQN with noisy exploration. The Axial CNN branch processes the cosine-token matrix, while the vector branch processes the Gram-Schmidt profile and scalar features. The reward function mainly encourages shortening the first basis vector, while also considering orthogonality improvement and computational cost.

The implementation combines C++/fpLLL, Rust and Python/PyTorch. The C++ backend performs high-precision LLL and local BKZ operations, the Rust layer manages basis matrices and extracts features, and the Python layer trains the DRL agent. This multi-language design allows exact lattice-reduction operations and neural-network training to work together.

The experimental results show that the proposed DRL-BKZ framework can reduce the Norm/GH ratio compared with the initial LLLed basis. The method also improves the logarithmic orthogonality defect and maximum cosine value in most tested dimensions. The cosine heatmaps further show that DRL-BKZ can weaken broad high-correlation regions, although some residual local hotspots remain.

Overall, this thesis demonstrates that DRL-based local BKZ scheduling is a feasible adaptive approach for lattice basis reduction. The method is not yet a complete replacement for classical BKZ, but it provides a useful proof-of-concept showing that learning-based scheduling can improve basis quality under a limited reduction budget. Future work can further strengthen this framework through larger experiments, complete ablation studies, multi-dimensional training and more flexible block-selection strategies.

Bibliography

- [1] Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.
- [2] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [3] Yuanmi Chen and Phong Q Nguyen. Bkz 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [4] Sik Feng Cheong, Chu Wee Lim, and Yan Bo Ti. Investigating deep reinforcement learning for bkz lattice reduction. 2025.
- [5] Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, et al. Noisy networks for exploration (2019). *arXiv preprint arXiv:1706.10295*, 2018.
- [6] Nicolas Gama and Phong Q Nguyen. Predicting lattice reduction. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 31–51. Springer, 2008.
- [7] Vlad Gheorghiu, Sergey Gorbunov, Michele Mosca, Bill Munson, et al. Quantum-proofing the blockchain. *Blockchain Research Institute: University of Waterloo*, 2017.
- [8] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *Annual Cryptology Conference*, pages 3–22. Springer, 2015.
- [9] AK Lenstra, HW Lenstra, and L Lovász. Factoring polynomials with rational coefficients. *Math. ann.*, 261(4):515–534, 1982.
- [10] Daniele Micciancio and Shafi Goldwasser. *Complexity of lattice problems: a cryptographic perspective*, volume 671. Springer Science & Business Media, 2002.
- [11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

- [12] Phong Q Nguyen and Jacques Stern. The two faces of lattices in cryptology. In *International Cryptography and Lattices Conference*, pages 146–180. Springer, 2001.
- [13] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016.
- [14] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.
- [15] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical programming*, 66(1):181–199, 1994.
- [16] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [17] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [18] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [19] Ziyu Zhao and Jintai Ding. Practical improvements on bkz algorithm. In Shlomi Dolev, Ehud Gudes, and Pascal Paillier, editors, *Cyber Security, Cryptology, and Machine Learning*, pages 273–284, Cham, 2023. Springer Nature Switzerland.
- [20] Ziyu Zhao, Jintai Ding, and Bo-Yin Yang. Sieving with streaming memory access. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):362–384, 2025.